SharpeArena: The Point-in-Time (PIT) RL Environment for Trading Agents

Tiberiu Toca
General Liquidity
tibi.toca@gmail.com

Abstract

A benchmark scores trajectories; something has to produce them, and the producer is where trading evaluations break. We describe SharpeArena, a point-in-time (PIT) RL environment for trading agents built so that three producer failures are excluded at the environment's own interface, and policed beyond it. Look-ahead: the data layer exposes no operation that returns a future bar, and a deny-list guard covers harness tools. Nondeterminism: one Rust engine restricted to multiply, add, divide and max, with committed golden hashes pinning scenarios and tapes byte-for-byte across the native, WebAssembly and Python surfaces; byte-identity covers the Rust core and its bindings, while the Python probe layer is seeded-deterministic on one platform. Trajectory forgery: runs record only decisions, and replay against the frozen scenario recomputes the result byte-identically. Agents speak a governed JSON wire contract, scenarios are procedural over disjoint train, gap and held-out seed bands, and ranking is delegated to the SharpeBench kernel. Ten committed experiments, and three follow-ups they motivated, calibrate the suite's instruments. They are calibrations, not market findings: the suite's own realism gate fails the canonical configuration's tape on 23 of 24 certification runs, an opt-in volatility-clustering driver is the partial remediation, and certification is reported beside the results rather than required by them. On the market-making task, fixed-spread quoting pays a U-shaped regret against the closed-form Avellaneda-Stoikov reference policy, from 84.6 at the tightest quote to a statistically-zero minimum at the closed-form spread. The manipulation probe stays negative on its symmetric schedule under concave impact exponents 0.5 and 0.7, but an asymmetric schedule search finds the profit theory predicts: at exponent 0.5, slow accumulation followed by block liquidation earns an impact-attributable $+21.8 \times 10^{-4}$ (95% CI $[17.4, 26.2]$) with no followers, while every one of 45 linear-impact points is negative with exact time-reversal symmetry, so manipulation is profitable under concave impact with asymmetric schedules and never under linear. A predictability probe finds no undisclosed next-bar edge for an honest AR adversary, yet recovers 16 of 16 seeds in about one second when the public seed band is bounded to 2^{16} candidates; an opt-in sealed-seed derivation under a commit-reveal salt closes that hole, the same scan recovering 0 of 16 sealed seeds and the revealed salt replaying 16 of 16. An out-of-band oracle witness shows that rank-eligibility is attainable on every tier, that the every-seed pass^k gate binds at all ten attained boundaries, and that on the Calm tier an every-bar trader cannot pass on costs alone even with a perfect oracle. Replicating the ecology probe over eight seeds overturns its own single-seed headline: the shock-driven winner replacement observed at the first seed occurs in one of eight seeds.

Preprint.

1 Introduction

An eval is useless without an environment. The companion SharpeBench paper [Toca, 2026] argues that ranking trading agents on raw return over short windows measures luck, and supplies a deterministic scoring kernel that deflates for search breadth [Bailey and de Prado, 2014], gates on per-run reliability [Yao et al., 2024], and zeroes entries that bypass risk controls. But a scorer consumes trajectories, and every one of its guarantees is conditional on the trajectory being honest. Three failures upstream of scoring void any score. An agent that observes one future bar has an edge no deflation can remove; look-ahead bugs are a pervasive and usually invisible failure mode of backtests, alongside the selection bias the deflation literature centers [Bailey and de Prado, 2014]. An environment that is not bit-reproducible cannot support a leaderboard, because the same submission scores differently on different machines and no one can check anyone else's number. And a trajectory that cannot be recomputed from the environment that allegedly produced it is a self-reported claim, not evidence.

SharpeArena is the producer built against those three failures, and its design method is uniform: exclude each failure at the environment's own interface, then add a detection layer behind the structural guarantee for defense in depth. Look-ahead is prevented by the shape of the data layer, which has no API that returns a future bar, and additionally policed by a deny-list guard at the harness boundary; this is an interface guarantee, and Section 7 states what it does not cover, including in-band prediction of the public deterministic generator. Determinism is achieved by restricting the scenario generator's arithmetic to operations that are exact and identical on every platform, and additionally pinned by committed golden hashes checked in continuous integration on the native and WebAssembly builds. Trajectory honesty is achieved by recording only decisions and recomputing everything else, and additionally exercised by a test that doctors a trajectory and asserts the replay diverges.

The interface follows the pattern that made Gym the substrate of a decade of reinforcement-learning research [Brockman et al., 2016, Towers et al., 2024]: a small, stable boundary that outlives any single environment behind it. An agent is a program in any language that reads a JSON `Observation` and writes a JSON `Decision`. The contract is versioned, additive-only, schema-published and governed by a written deprecation policy, so a vendor can build against it once. Conform to the contract and you are scorable everywhere the SharpeBench kernel runs. Whether an ecosystem forms around this contract is an adoption question the paper does not answer; Section 7 records that no third party has yet submitted an agent.

Scope and contribution boundary. This paper claims the environment and its protocol: the interface-level leak-freedom, the cross-runtime determinism, the replay verifiability, the wire contract, the seed-band generalization protocol, and the probe layer, together with the calibration experiments that exercise them. The scoring semantics, the deflation prior and the unit bug in the pre-0.5.0 kernel are the companion SharpeBench paper's contributions [Toca, 2026]; F1 appears here only as validation that the producer-scorer separation catches scorer bugs. No agent is trained in this paper: every experiment uses fixed reference policies, so the results calibrate the instruments rather than demonstrate learned performance, and a learning baseline through the shipped Gymnasium adapter is named future work in Section 7.

Contributions. (1) A PIT RL environment for trading agents that is leak-free at its interface boundary, cross-runtime byte-deterministic in its Rust core, and replay-verifiable, with the enforcing code identified for each property. (2) A governed agent wire contract at version 1.0 with JSON Schemas, conformance fixtures and an additive-only evolution policy. (3) A procedural scenario generator with disjoint train, gap and held-out seed bands in the Procgen tradition [Cobbe et al., 2020], making generalization a measured quantity. (4) A task suite that includes a market-making environment shipping the closed-form Avellaneda-Stoikov policy [Avellaneda and Stoikov, 2008, Guéant et al., 2013] as a reference, so agents are scored on regret against a fixed analytical baseline rather than against each other. (5) A diagnostic layer that probes realism, manipulation payoff

boundaries, adverse-selection markouts, population ecology and generator predictability, including an opt-in concave-impact path that leaves the canonical linear golden path byte-identical, and an opt-in sealed-seed derivation that hides the held-out seeds behind a commit-reveal salt while keeping the disjointness proof public. (6) Ten executed experiments and three follow-ups (Section 5) in which every number is produced by a committed script listed in Section A.

What the evidence shows. The ten findings are instrument calibrations produced on tape that the suite's own realism gate declines to certify at the canonical configuration (Section 5.5); that framing governs every number below. On the market-making task the regret metric behaves as its construction implies: fixed-spread quoting pays a U-shaped regret against the analytical reference (Section 5.3). Cross-regime transfer costs the calm-selected reference nearly its entire DSR (Section 5.4). Linear-impact pumping is unprofitable as theory predicts, and under concave exponents 0.5 and 0.7 the same symmetric schedule remains negative (Sections 5.6 and 5.7); the asymmetric positive control then finds what the theory permits and only that: at exponent 0.5 a slow-accumulate, block-liquidate round trip earns $+21.8 \times 10^{-4}$ with a CI clear of zero and no followers, its mirror loses heavily, and all 45 linear-impact points are negative (Section 5.8). A causal AR adversary gains only the generator's disclosed Calm momentum and no stressed-tier edge, whereas a bounded 2^{16} seed band is inverted in about one second with 16 of 16 recoveries (Section 5.12); sealing the band behind a commit-reveal salt drops the same scan to 0 of 16 while the revealed salt replays 16 of 16 (Section 5.13). The adverse-selection, failure-taxonomy and replicated ecology probes expose their own limitations rather than hiding them (Sections 5.9 to 5.11). Under the corrected companion kernel no reference policy is rank-eligible because none passes every seed (Section 5.1), yet eligibility is attainable: an out-of-band oracle opens the gates on every tier, the every-seed pass^k leg is what binds at every boundary, and on Calm an every-bar trader cannot pass on costs alone even with a perfect oracle (Section 5.2).

2 Design principles

Every decision in the environment traces to one of seven principles, stated here so the rest of the paper reads as their consequences.

Leak-free at the interface. The environment owns the time cursor, and the data layer exposes no operation that returns a bar after it. An agent cannot peek through the interface because there is nothing to call, not because a rule forbids it. Detection layers exist, but they sit behind the structural guarantee, never in place of it. This is a guarantee about the interface's shape, not about information: what an agent might infer in-band from the observations the interface does return, given a public deterministic generator, is a separate question, treated in Section 7.

Deterministic across runtimes. A published number must be byte-identical on any platform and any surface. The determinism-critical core is one Rust engine; its scenario transform uses only multiply, add, divide and max, because `ln` and `exp` differ across libm implementations. Golden hashes committed in the source pin the outputs, and the WebAssembly build asserts equality with the native engine rather than assuming it.

Recompute, do not trust. A trajectory is evidence only if a third party can recompute it. Runs record the agent's decisions and nothing derived; replay against the frozen scenario reproduces returns byte-for-byte, and a tampered trajectory reproduces something else.

The contract is the product. The agent interface is a tiny JSON surface that evolves additively under a written governance policy, with schemas and conformance fixtures as the authoritative artifacts. An interface that moves under its implementers cannot be built against; stability under governance is what makes the contract worth conforming to.

Generalization is measured, not presumed. Scenarios are procedural functions of an integer seed, in the Procgen model [Cobbe et al., 2020], and the protocol fixes disjoint train, gap and held-out seed bands. Overfitting becomes one number, the train-minus-test deflated Sharpe, instead of a suspicion.

The environment produces; the kernel judges. SharpeArena computes no rank. Scoring is delegated to the SharpeBench kernel [Toca, 2026], so the deflation, reliability and process gates are specified once, in one place, and the environment cannot quietly disagree with the scorer about what a good run is.

Distrust the simulator too. A synthetic market that pays manipulation without bound, or that lets makers profit from informed flow, or that emits Gaussian tape, is an answer key for the wrong exam. The probe layer treats the simulator as a subject: realism certification against the stylized facts [Cont, 2001], payoff-boundary sweeps for manipulation, markout decomposition for adverse selection, and ecology runs that ask which strategies survive contact with each other. At this revision the probes report findings against the environment as often as for it: the realism gate fails the canonical tape (Section 5.5) and the adverse-selection levels sit on the wrong side of this principle's own standard (Section 5.9).

One consequence is worth stating before the experiments. The first three principles are properties of committed code, checkable today by running the listed tests, and this paper claims them as such. The empirical section's claims are of a different kind: eight core findings were produced by the original committed evidence run (`sharpearena` 0.8.0 over SharpeBench 0.5.0), two reviewer-motivated probes were added on the v0.11.0 source tree, and three follow-ups (a positive control, an existence witness and sealed seeds) on the post-0.11.1 tree. All of them calibrate instruments on this generator rather than certify any market property.

3 The environment

SharpeArena is a Rust workspace of three crates. The crate `sharpearena` holds everything determinism-critical: the stepping engine, the wire contract, the scenario generator, the limit-order-book matching engine, mandates and the execution-noise model, built on the published SharpeBench engine crates rather than a vendored copy. The crate `sharpearena-wasm` compiles the same kernel to WebAssembly and tests it against the native engine. The crate `sharpearena-py` binds the engine into a Python package that adds the ecosystem adapters: Gymnasium [Towers et al., 2024], PettingZoo [Terry et al., 2021], Minari [Farama Foundation, 2024], a `verifiers` training environment, and an MCP server. The adapters are thin by policy; anything that must be byte-identical lives in Rust.

3.1 Point-in-time observation, and the guard behind it

The engine owns the time cursor. On each step it emits a `MarketObservation` whose per-symbol snapshot carries a trailing window of closes up to the decision date, with optional fundamentals and news channels derived from that same trailing window. There is no API on the data layer that returns a bar after the cursor, so a policy cannot observe a future bar: the property is the shape of the interface, not a rule. It is also only a property of the interface: it bounds what can be requested, not what can be inferred from the trailing window a request returns (see Section 7). A property test iterates every scenario tier and asserts that no surfaced window extends past the cleared bar and that each window is a suffix of the true history.

One layer out, at the agent-harness boundary, sits `LookaheadGuard`, a refusal layer for tools and policies that try to read what the environment will not give. It carries a deny-list of named operations, each mapped to the reason it is refused: reading the raw dataset (the answer key), slicing past the current index, peeking the next bar, or feeding the scenario seed to the policy as a feature, which would identify the episode and trivialize generalization. Substring tells catch novel operation names, and an index check bounds any point-in-time slice: reading an index strictly greater than the current bar raises. A research escape hatch exists as an explicit environment variable, so relaxing the guard is a recorded decision rather than an accident. The guard is defense in depth behind the structural guarantee: a harness that bypasses it still cannot make the environment leak.

3.2 Determinism across runtimes

A scenario is a pure function of one 64-bit seed. The generator's price transform uses only multiply, add, divide and max; it never calls `ln` or `exp`, which differ across libm implementations, so a generated panel is byte-identical on any platform and any runtime. The commitment is pinned, not assumed: the source commits FNV-1a fingerprints of canonical generated scenarios and of a scripted order sequence's fill tape. Tests recompute the hashes on every commit, and WebAssembly parity tests require native-identical returns, traces and scenarios. The generator is deliberately public and deterministic; that makes the hashes checkable and future bars computable from a recovered seed. Section 5.12 measures the boundary: a causal AR adversary finds no undisclosed stressed-tier edge, but a public bounded band of 2^{16} seeds is exhaustively inverted from one observed bar in about one second. Held-out evaluation therefore requires high-entropy unpublished seeds, not merely disjoint public bands.

3.3 Recompute-from-decisions tamper evidence

A rollout trace is a versioned JSONL artifact that records the agent's decisions and rejects anything leaky: the writer refuses values that would embed future data in the record. Everything else, returns included, is recomputed at verification time by replaying the decisions against the frozen scenario, and because the engine is deterministic the recompute is byte-identical. The npm package's test suite exercises the adversarial case directly: it captures a trajectory, doctors every order's target weight, replays both, and asserts the tampered artifact cannot reproduce the honest returns. An agent cannot lie about its performance to anyone willing to run the replay.

3.4 Procedural scenarios and disjoint seed bands

Scenario families follow Procgen's integer-seed-interval model [Cobbe et al., 2020]: Calm, Hard and Extreme volatility-and-jump tiers, a cointegrated-pairs family with a genuinely mean-reverting spread, and a regime-shift family that hands a trend regime over to a whipsaw. The Rust core exposes `train_test_split`, which carves a disjoint test family (two separated integer intervals, with the disjointness asserted in tests) from a bounded train interval and refuses an unbounded one, and `cross_regime_split`, which holds the seed band fixed while swapping the regime, a strictly stronger robustness probe than a within-tier split. The evaluation protocol fixes the canonical bands: train seeds $[0, 256)$, a gap of ten thousand seeds that is never sampled, and a held-out test band $[10256, 10512)$. The Python layer additionally reserves an absolute evaluation namespace starting at seed 10^6 and commits a named held-out regression set inside it, with a disjointness guard asserting the set never touches the train band. Overfitting is then a measurement: `generalization_gap` scores the same policy on train and held-out bands and reports the differenced deflated Sharpe, and `cross_regime_transfer` reports the in-distribution minus zero-shot out-of-distribution gap over identical seeds, which is zero by construction when the regimes coincide.

3.5 The task suite and the closed-form reference policy

Beyond the canonical position-trading environment, the suite ships a simplex-allocation portfolio task, a VWAP/TWAP implementation-shortfall execution task, and two market models with genuine microstructure: an endogenous shared-book market where aggregate agent flow moves the cleared price through a Kyle permanent-impact term [Kyle, 1985] and an Almgren-Chriss temporary-impact term [Almgren and Chriss, 2001], and a deterministic limit-order-book market with integer ticks, price-time priority, a single-price call-auction uncross and a read-only walk-the-book sweep-cost query.

Both market models clear once per step at a single price, so the trading mechanism is closer to a frequent batch auction [Budish et al., 2015] than to a continuous-time limit order book. The competition margins that continuous markets create, latency races, queue-position value, sniping of stale quotes, do not exist in the environment, and no maker-taker fees or rebates are modeled: every

Sharpe, regret and markout in Section 5 is gross of fees. The restriction is deliberate, it keeps clearing deterministic and byte-reproducible, and it makes the environment a batch-auction laboratory rather than a latency one, but it is a scope restriction and results should be read inside it.

The market-making task is the suite's calibration instrument. It implements the Avellaneda-Stoikov model [Avellaneda and Stoikov, 2008] and ships the model's closed-form quoting policy beside the environment: reservation price $r = s - q\gamma\sigma^2\tau$ and half-spread $\delta^* = \gamma\sigma^2\tau/2 + \gamma^{-1}\ln(1+\gamma/\kappa)$, skewed by inventory. This closed form is the source model's asymptotic approximation, not an exact optimum: the exact treatment of the same problem is given by Guéant et al. [2013], and no proof is offered that the policy is optimal for the implemented reward, which adds an inventory cap, a running inventory penalty and a terminal liquidation charge the source model lacks. We therefore call it the closed-form reference policy. Both the environment and the policy are pure functions of the same frozen parameter set, so the regret metric `mm_regret` runs the candidate and the reference on identical seeds and reports the mean reward gap; the reference scores zero against itself by construction, which fixes the metric's zero point without claiming the zero point is unbeatable. The model also assumes fills that do not select against the maker, which sits in deliberate tension with the adverse-selection probe of Section 5.9. Gym-style market-making environments built on the same model family exist [Jerome et al., 2023]; what this task adds is the shipped reference policy and the regret-not-PnL scoring convention inside the verifiable-producer stack.

3.6 Mandates and deferred claims

Each scenario can sample a trading mandate, long-only, market-neutral, drawdown-capped or beat-a-benchmark, and the objective is conditioned on it: `mandate_breach` detects structural violations from the realized weights and returns, and the training rubric penalizes wrong-objective behavior rather than merely not rewarding it. A deterministic failure taxonomy classifies every episode into a single worst-case-first disposition, cascade-wiped before bankrupt before stopped-out before mandate breach before clean, and a suite-level rollup tallies the distribution with a schema that always carries every mode.

Questions that resolve after the episode get the same structural treatment as look-ahead. A `DeferredDesk` is the only object the agent touches: its entire state is a list of claims and one integer clock advanced by the harness, it holds no dataset and no environment reference, and it computes no score, so there is no code path from a claim to its resolving datum. Resolution is a free function over claims and outcomes that refuses any outcome available at or before the claim's commit bar and any outcome predating the stated horizon. The agent cannot see the answer at commit time because the object it holds cannot produce one.

3.7 The probe layer: distrusting the simulator

A synthetic market can flatter agents in ways real markets do not, so the suite probes itself.

Realism. A stylized-facts battery computes excess kurtosis, absolute-return autocorrelation, gain/loss skew and aggregational Gaussianity in the sense of the classical catalogue [Cont, 2001], plus a timescale-asymmetry statistic in the sense of Zumbach [2009] and a Fano burstiness factor that is the authors' own diagnostic (the variance-to-mean ratio of large-move exceedance counts per window, equal to 1 under Poisson arrivals, with a super-Poisson bound chosen to demand clustered large moves), over a generated panel; `certify_realism` grades the panel against directional believability bounds. The gate's role at this revision must be stated plainly: certification is *reported* beside the results, it is not a precondition for the leaderboard or for the findings of Section 5, and the canonical configuration currently fails it (Section 5.5). The intended endgame is either a generator revision that passes at tier level (the opt-in volatility-clustering driver of Section 5.5 is the first step) or leaderboard claims conditioned on certification; until then the gate is a diagnostic that keeps the failure visible. We chose to ship verifiability first and realism second because a realistic but unverifiable tape cannot anchor a leaderboard, while a verifiable and admittedly unrealistic one can still calibrate instruments; the ordering is an argued choice, not an oversight.

Manipulation. A crude push-and-unwind schedule, the classic trade-based manipulation of Allen and Gale [1992], is scored against its own zero-impact reference: the live and reference runs share the seed and follower population, and differ only in impact. A boundary sweep varies one parameter and a size-response sweep asks whether payoff is bounded. Linear permanent impact makes the baseline a theory consistency check [Huberman and Stanzl, 2004, Gatheral, 2010]. An opt-in exponent replaces λQ with $\lambda \operatorname{sign}(Q)|Q|^{\kappa}$ on separate entry points, leaving $\kappa = 1$ and every published golden on the original arithmetic path; Section 5.7 evaluates $\kappa \in \{0.5, 0.7\}$ on the symmetric schedule, and Section 5.8 searches asymmetric schedules and finds the profitable round trip theory permits at $\kappa = 0.5$.

Adverse selection. Informed meta-order flow is routed against a maker roster, with fills struck at the pre-move mid so the maker's markout carries the informed trader's edge. The mechanism the probe operationalizes is the classical one [Glosten and Milgrom, 1985], with one deliberate simplification: the price path is exogenous and quotes do not widen in equilibrium response to toxicity, so the probe measures markout accounting against drift, not equilibrium adverse selection. The design is a paired control: the informed leg sides child orders on the alpha, the uninformed leg sides the identical flow on a coin while holding the price path fixed, and the comparison reports mean markout per filled unit at each horizon. If the informed leg does not mark out worse, the module's own numbers are declared noise.

Ecology. A replicator dynamic in the market-ecology tradition [Farmer, 2002, Lux and Marchesi, 1999] runs a population of strategies over generations: fitness earned in a shared-book market where a species that gains share becomes a larger fraction of the flow everyone else fills against, seat allocation from population shares, extinction below a threshold, an innovation operator that breeds parameter-perturbed variants, and shock schedules that rotate regimes or scale impact to simulate liquidity droughts. The control schedule holds the environment steady, so any claim about a shock has a baseline to beat.

4 The agent contract and its governance

An agent is an external program in any language that reads a `MarketObservation` as JSON and replies with a `Decision` as JSON. The harness sends an observation, the agent returns a decision, repeat. Everything else in this paper exists to make that loop trustworthy; this section is the loop's promise of stability.

4.1 Authoritative artifacts

The contract has four authoritative artifacts, all committed. `CONTRACT_VERSION`, currently `1.0`, is the single source of truth for the wire version and lives in the contract module of the core crate. Two JSON Schemas, draft 2020-12, define `MarketObservation` and `Decision` for non-Rust implementers. A conformance kit of fixture documents, covering the normal multi-symbol case, a single symbol, an empty-portfolio start, signed-weight shorting and a legacy decision shape, is exercised by a committed conformance test. And a governance document states the evolution rules in writing.

4.2 Additive-only evolution

The contract evolves additively. The only backwards-compatible change is a new field that is optional with a default, so an agent written against an older version keeps parsing newer observations and the harness keeps parsing older decisions. The optional fields are absent from the schemas' `required` lists by construction. Removing, renaming or retyping a field, making an optional field required, and removing or renaming an action variant are all defined as breaking and forbidden on the v1 surface. A breaking change never mutates the v1 types in place; it ships a parallel namespace with its own schemas and its own major version, and the two run side by side through a published deprecation window. Adding a new action variant is itself classified as breaking, because consumers

that exhaustively match would misparse it, so it also goes through the parallel namespace rather than an additive bump.

4.3 Version semantics

`CONTRACT_VERSION` versions the wire shape only and is deliberately decoupled from the crate and package versions, which move on their own release cadence. A major bump is a breaking change shipped as a parallel namespace. A minor bump is reserved for a batch of additive fields significant enough to advertise; a single additive field needs no bump at all, because existing agents are unaffected by definition. A package release never, on its own, bumps the contract version.

4.4 Why the contract can be trusted without trusting the host

The governance promise is social; the properties of Sections 3.1 to 3.3 are technical, and together they remove the host from the trust equation for everything the byte-identity guarantee covers. That coverage has a boundary worth stating where the guarantee is advertised: the Rust core, its WebAssembly and Python bindings, and the engine outputs behind F1 through F3 are pinned to the byte; the Python evidence layer behind the remaining probes is deterministic in its seeds but sits outside the golden-hash path (Section 7). Within the covered surface, a leaderboard entry can be recomputed from the frozen seed and submitted decisions on any tested platform. The scoring rules are defined independently by the SharpeBench kernel's deflated Sharpe, reliability and process gates [Toca, 2026, Bailey and de Prado, 2014].

5 Experiments

Ten findings, eight core calibrations plus two reviewer-motivated probes, and three follow-ups that close questions the ten left open: an existence witness for rank-eligibility (Section 5.2), an asymmetric positive control for the manipulation probe (Section 5.8) and sealed evaluation seeds (Section 5.13). Each subsection states the entry point, seeds and quantity measured, then reports numbers from committed JSON and figures. The exact commands are listed in Section A. The original evidence records SharpeBench 0.5.0 under `sharpearena` 0.8.0; the concave-impact and predictability extensions were produced on the v0.11.0 source tree, and the three follow-ups on the post-0.11.1 tree, each with its configuration serialized beside its results.

Three scoping statements govern the whole section. First, the findings are instrument calibrations on a generator whose canonical tape the realism gate of Section 5.5 declines to certify (1 of 24 runs passes the full conjunction); certification is reported beside the results, not required by them. Second, the byte-identity guarantee covers engine outputs behind F1 through F3 and the canonical linear path; Python evidence reductions are seed-deterministic on one platform but outside the golden-hash path. Third, the core evidence uses the explicitly stated train-band seeds except F3, which exercises the split; the predictability probe uses a separate 50,000-start band and reports why a bounded public band is unsafe. Dispersion is reported throughout: F1 and F3 use 2,000 seed bootstrap resamples, while F2, F5 and F6 use t-based 95% intervals over committed vectors.

5.1 F1: Pipeline validation under the corrected kernel

The reference-policy field (flat, equal-weight long, momentum, minimum-variance, maximum-Sharpe, fractional-Kelly volatility targeting) is rolled over sixteen scenario seeds in each of the Calm, Hard and Extreme tiers via `run_baselines`. Each policy's pooled return series is scored by the SharpeBench `score_run` kernel with the declared trial count equal to the field size, and each row carries a seed-paired bootstrap 95% confidence interval on the deflated Sharpe (2,000 resamples).

This experiment validates the producer-scorer pipeline end to end; the scorer-side discovery it reproduces belongs to the companion paper. Under the pre-0.5.0 kernel this board was all zeros: every baseline scored a deflated Sharpe of 0.0000 on every tier. That was not strictness. The kernel

Table 1: Baseline leaderboard under the corrected kernel: deflated Sharpe (DSR) and pass^k rate per tier, 16 seeds each. No policy is rank-eligible on any tier: eligibility requires the per-run gate to pass on every seed (a pass^k rate of exactly 1.00), and the best rate is 0.75.

	Calm		Hard		Extreme	
Policy	DSR	pass^k	DSR	pass^k	DSR	pass^k
kelly_vol_target	1.0000	0.75	0.0277	0.31	0.0029	0.12
equal_weight_long	1.0000	0.62	0.1114	0.31	0.0234	0.06
min_variance	0.9849	0.44	0.0024	0.19	0.0243	0.06
flat	0.0007	0.00	0.0007	0.00	0.0007	0.00
momentum	0.0000	0.00	0.0000	0.00	0.0000	0.06
max_sharpe	0.0000	0.06	0.0000	0.00	0.0000	0.00

applied its annualized 0.5 deflation prior per period without conversion, which set the bar near an annualized Sharpe of 18 on daily bars, a level nothing clears. Finding and fixing that unit bug is the companion SharpeBench paper's Finding 1 [Toca, 2026] (the derivation: an annualized prior applied per period, corrected by converting once at 252 periods per year); what this paper claims is only that the producer-scorer separation surfaced it, because trajectories produced here could be re-scored under both kernels and the discrepancy attributed to the scorer alone.

Table 1 and Figure 1 show the corrected board. On Calm, drift is free deflated Sharpe: `kelly_vol_target` and `equal_weight_long` saturate the DSR at 1.0000 (bootstrap CIs $[0.9798, 1.0000]$ and $[0.8343, 1.0000]$) and `min_variance` reaches 0.9849, though with a near-vacuous interval of $[0.0264, 1.0000]$. Yet nothing on the board is rank-eligible, because no policy passes the per-run gate on every seed; the best pass^k rate is 0.75. On Hard the best DSR collapses to 0.1114 and on Extreme to 0.0243, and the long baseline's pass^k rate degrades monotonically with stress (0.62, 0.31, 0.06). `momentum` and `max_sharpe` lose money outright on every tier (mean per-bar returns down to -1.6×10^{-3} on Extreme).

The reading: eligibility is a conjunction, and each leg catches what the other misses. Deflation alone would crown drift on Calm; pass^k alone would miss the overfit breadth the deflation discounts. The earlier all-zeros board looked strict but was measuring a unit error. The corrected board is the honest zero: the number a real agent has to beat is a positive, deflated Sharpe that survives every evaluation seed, and no reference policy produces one. One caveat the board cannot answer on its own: no reference policy is rank-eligible, so the board alone does not show that the eligibility set is non-empty for this task family. Section 5.2 answers that by construction, injecting an out-of-band oracle signal of controlled strength and locating the strength at which the gates first open; the region is non-empty on every tier, and the per-run gate that fails every baseline here is the leg that binds there too.

5.2 Rank-eligibility existence witness: the acceptance region is non-empty, and here is its boundary

Section 5.1 reports that no reference policy is rank-eligible on any tier and that even `flat` fails the per-run gate on all 16 seeds, and it leaves open whether the eligibility set is non-empty at all for this task family. That is the right objection: a gate that nothing can pass is not strict, it is vacuous, and "the number an entrant has to beat" would then be undefined rather than high. This fragment answers by construction, the way the SharpeBench kernel's own pass witness does. We inject a known edge of controlled strength and locate the strength at which the gates first open.

The out-of-band oracle, and why it is not an entrant. The generator is deterministic, so the true next-bar return of every symbol is recoverable in-script by replaying the same seed one bar ahead with flat decisions; the canonical position-trading tape is policy-invariant, which the script asserts on every rollout. No entrant can do this through the observation interface (Section 5.12 measures what an honest adversary recovers), and the oracle is not scored on any board. It exists only to place a signal of known quality in a policy's hands: $\text{signal}_t = s\, z_t + \sqrt{1 - s^2}\, \varepsilon_t$, with z_t the standardized

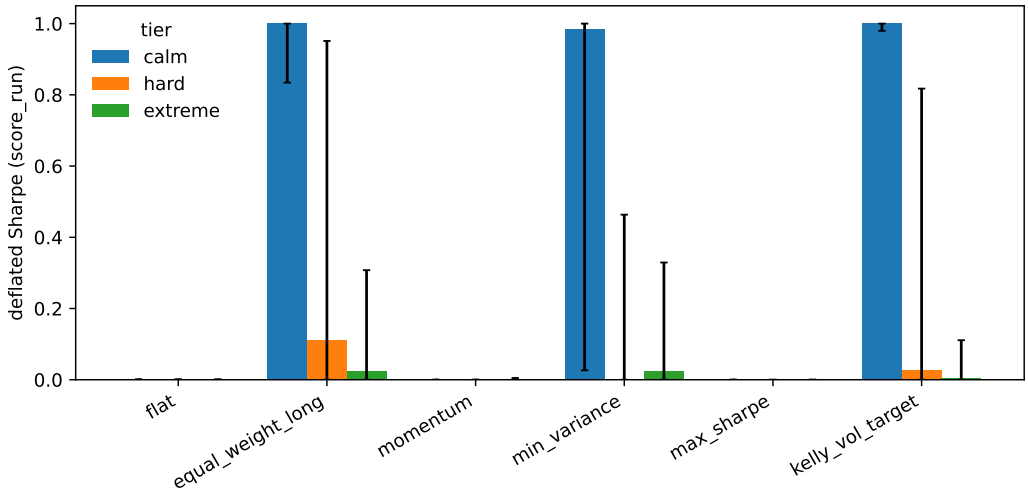


Figure 1: Deflated Sharpe per baseline per tier with seed-paired bootstrap CIs. Calm saturates the long baselines; Hard and Extreme collapse them toward zero.

true next-bar return, ε_t seeded standard normal noise, and s therefore the correlation between signal and truth, from a pure noise trader at $s = 0$ to omniscience at $s = 1$. Two policies consume the signal. The canonical `sign_follow` trades $\mathrm{sign}(\mathrm{signal})$ at $1/n$ gross per symbol every bar, the momentum baseline's exposure convention. The turnover-aware `deadband_hold` trades the sign only when $|\mathrm{signal}| > 1$ and otherwise holds its previous position. Every strength is scored exactly as the baselines are: the per-run gate on each of 16 seeds through `score_run`, the pooled series through `score_run` for the deflated Sharpe, the bootstrap p-value and the kernel's `rank_eligible` flag, a seed-paired bootstrap CI on the pooled DSR, and the F1 deflation footprint ($n_{\mathrm{trials}} = 6$). Eligibility is the paper's rule: the per-run gate passes on every seed (pass^k rate exactly 1.00) and the kernel's conjunction holds on the pooled run (DSR ≥ 0.95, bootstrap $p < 0.05$, process and mandate clean). A coarse sweep over 13 strengths is refined by bisection to a resolution of 0.005 in s. The primary band is the canonical held-out band (16 seeds, 10,000-seed gap, as in Section 5.4); the F1 table band is rerun as a cross-check.

Result: eligibility is attainable on every tier, and the every-seed gate is what binds. Table 2 gives the boundaries. With the canonical sign-follower the region opens at $s \in [0.734, 0.738]$ on Hard (signal accuracy 0.71, mean 7.8 bp per bar) and at $s \in [0.338, 0.341]$ on Extreme (accuracy 0.57, 27 bp per bar). On Calm it never opens: even the perfect oracle passes the per-run gate on only 11 of 16 seeds (minimum seed PSR 0.026), because a policy that flips every symbol every bar pays roughly 14 bp per bar in round-trip costs (that is the $s = 0$ noise trader's loss), while the omniscient sign of a Calm bar is worth only 2.7 bp net. The turnover-aware variant removes that ceiling: it is eligible on Calm from $s \in [0.903, 0.906]$ (accuracy 0.88, 6.4 bp per bar), on Hard from $s \in [0.472, 0.475]$ (accuracy 0.62, 9.7 bp) and on Extreme from $s \in [0.284, 0.288]$ (accuracy 0.55, 25 bp). On the F1 table band the boundaries sit at $s \in [0.800, 0.803]$ (Hard) and $[0.425, 0.428]$ (Extreme) for the sign-follower and at $[0.713, 0.716]$, $[0.625, 0.628]$ and $[0.328, 0.331]$ for the deadband variant, the same ordering with band-luck shifts of the size Section 5.4 calibrates. At all ten boundaries that exist the binding gate is the same one: the per-run PSR bar on every seed. The pooled legs open well before it. On the held-out band with the deadband variant the DSR bar is cleared at $s = 0.40, 0.30$ and 0.20 (Calm, Hard, Extreme) and the bootstrap at 0.40, 0.20 and 0.05, whereas the every-seed gate waits until 0.906, 0.475 and 0.288; the sign-follower shows the same order, DSR bar at 0.70 and 0.25 against every-seed at 0.738 and 0.341 on Hard and Extreme. In every case the last seed to pass is the one with the least favourable path, and near the boundary the gate is brittle in the way a

Table 2: Existence witness: the signal strength s at which rank-eligibility begins, per tier, policy variant and seed band, with the gate still failing on the ineligible side. Strength is the correlation between the oracle-mixed signal and the true next-bar return. "none" means no strength up to and including $s = 1$ is eligible.

Variant	Band	Calm	Hard	Extreme
`sign_follow`	held-out	none (11/16 at $s = 1$)	$[0.734, 0.738]$	$[0.338, 0.341]$
`sign_follow`	F1 table	none (11/16 at $s = 1$)	$[0.800, 0.803]$	$[0.425, 0.428]$
`deadband_hold`	held-out	$[0.903, 0.906]$	$[0.472, 0.475]$	$[0.284, 0.288]$
`deadband_hold`	F1 table	$[0.713, 0.716]$	$[0.625, 0.628]$	$[0.328, 0.331]$
Binding gate	all	per-run PSR ≥ 0.90 on every seed (pass^k)		

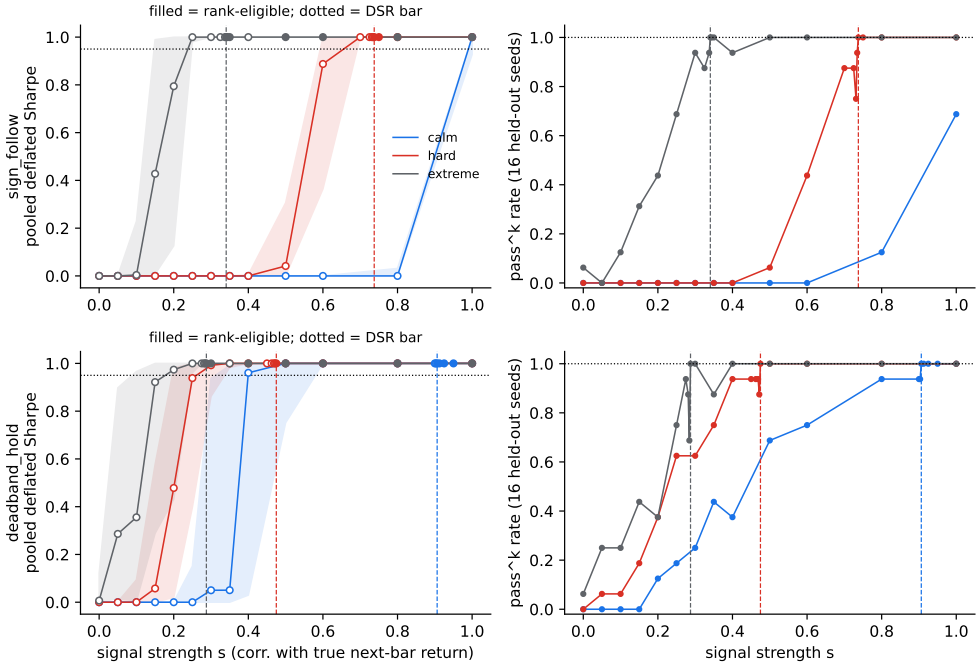


Figure 2: Existence witness on the held-out band, one row per policy variant. Left: pooled deflated Sharpe against signal strength with seed-paired bootstrap CIs; filled markers are rank-eligible, dashed lines mark the boundary, the dotted line is the DSR bar. Right: pass^k rate over 16 seeds. The pooled DSR saturates long before the every-seed gate opens.

conjunction over 16 draws must be: on Extreme, $s = 0.40$ fails on one seed (PSR 0.880) while 0.35 and 0.50 pass, so the boundary is a band, not a point, and the table reports it as one.

Reading. Three things follow. The acceptance region is non-empty on every tier, so the gates are strict rather than vacuous, and the zero on the baseline board is a measurement of the baselines. Its boundary is far from the trivial policies: an entrant needs the equivalent of a 0.62 to 0.74 directional hit rate on Hard bars depending on how it spends turnover, and it needs that on every one of 16 held-out paths, not on average. And the per-run gate that the devil's advocate suspected of hiding an unstated requirement does contain one, but it is documented in the kernel and it is a positive-edge requirement, not an activity requirement: PSR ≥ 0.90 against a zero-Sharpe benchmark on each run, which a zero-dispersion series fails at PSR $= 0.5$ by construction. `flat` therefore fails 16 of 16 not because doing nothing is penalized but because doing nothing carries no evidence of edge, and the same rule refuses a clone of the benchmark. The Calm ceiling on the sign-follower is the environment's cost model at work: on the low-volatility tier the per-bar edge of a perfect sign is smaller than the cost of trading it every bar, so eligibility there requires a policy that rations turnover,

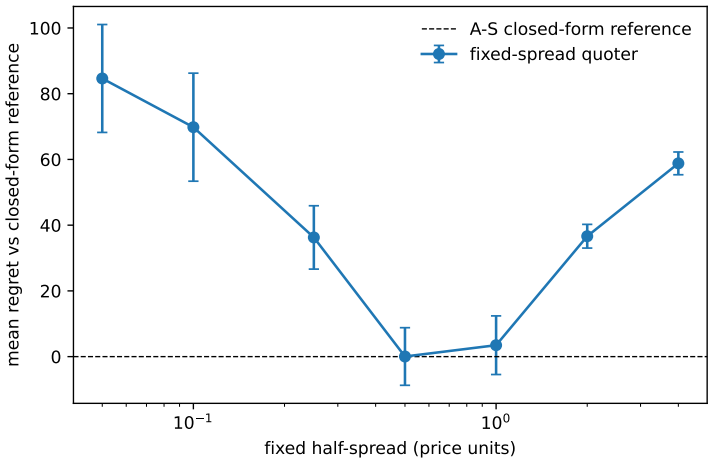


Figure 3: Mean regret versus the closed-form Avellaneda-Stoikov reference policy over 16 paired episodes, with t-based 95% CIs. Fixed-spread quoting is U-shaped in the half-spread; the reference defines zero.

which is what a real entrant would have to learn. The oracle is not that entrant; it is the ruler the entrant will be measured against.

5.3 F2: Regret against the closed-form reference policy

On the Avellaneda-Stoikov market-making task, `mm_regret` runs each candidate quoting policy and the closed-form reference policy (Section 3.5) on the same sixteen seeded episodes ($\sigma = 2.0$, $\gamma = 0.1$, $\kappa = 1.5$, 200 steps) and reports the mean reward gap to the reference. Candidates are the reference itself and fixed-spread quoters over a grid of half-spreads. The evidence commits the per-episode regret vector per grid point; intervals are t-based 95% CIs over the sixteen paired episodes.

The reference's regret is 0.0 by construction, which confirms the plumbing: the metric's zero point is the reference, not a claim of unbeatability. The fixed-spread curve is U-shaped (Figure 3): regret is 84.6 (95% CI $[68.2, 101.1]$) at a half-spread of 0.05 (quoting too tight, filled constantly, run over by inventory), falls to a minimum of 0.037 at 0.5, and rises again to 36.6 ($[33.0, 40.2]$) at 2.0 and 58.8 ($[55.3, 62.3]$) at 4.0 (quoting too wide, earning nothing). The minimum deserves its interval: the 0.5 point's CI is $[-8.7, 8.8]$ and the 1.0 point's is $[-5.4, 12.4]$, so the best fixed spreads are statistically indistinguishable from the reference on sixteen episodes, while the tight and wide extremes are separated from it by wide margins. The instrument resolves the shape of the curve, not the sign of its minimum.

The point of shipping the reference is that agents on this task are scored on regret, not raw PnL. A market-making leaderboard ranked on PnL rewards whichever policy drew the friendliest flow; regret against a fixed analytical reference on the same episodes removes that luck axis.

5.4 F3: Generalization gap and cross-regime transfer

`generalization_gap` scores the default reference policy (equal-weight long) on disjoint train and held-out seed bands (16 seeds each, separated by a 10,000-seed gap) within each tier. `cross_regime_transfer` then holds sixteen seeds fixed and varies the regime, producing the full three-by-three transfer matrix whose diagonal is zero by construction, because identical regimes reuse byte-identical environments. One framing note before the numbers: the policy is fixed and parameter-free, so nothing is trained, "source regime" below means the regime the policy is nominally selected on, and for such a policy the matrix reduces to pairwise differences of three per-tier deflated Sharpes. What F3 measures is the instrument, how much score a regime switch moves for a

Table 3: Left: within-tier generalization gap (train minus test deflated Sharpe) for the fixed equal-weight reference. Right: cross-regime transfer gap (in-distribution minus zero-shot out-of-distribution DSR), rows source regime, columns scored regime; the diagonal is zero by construction, and the Hard↔Extreme cells (± 0.096) sit below the 0.34 seed-noise calibration and are not interpreted.

Tier	Train DSR	Test DSR	Gap
Calm	1.0000	0.9988	$+0.0012$
Hard	0.1231	0.4618	-0.3388
Extreme	0.0269	0.1280	-0.1012

	Calm	Hard	Extreme
Calm	0.000	$+0.877$	$+0.973$
Hard	-0.877	0.000	$+0.096$
Extreme	-0.973	-0.096	0.000

Table 4: Stylized-facts certification per tier: mean fact value over 8 seeds, per-fact pass counts, and the all-facts pass rate. Calm is near-Gaussian by design and fails the fat-tail facts; the stressed tiers pass them.

	Excess kurtosis		$\lvert r\rvert$ autocorr		Fano factor		All facts
Tier	mean	pass	mean	pass	mean	pass	pass rate
Calm	-0.97	0/8	$+0.0002$	5/8	1.19	7/8	0.000
Hard	$+16.30$	8/8	-0.0107	1/8	0.85	0/8	0.000
Extreme	$+11.38$	8/8	-0.0118	1/8	0.79	1/8	0.125

fixed reference, not transfer learning. The evidence commits per-seed return series per cell; CIs are seed-resampled percentile bootstrap ($B = 2000$), seed-paired across regimes for the matrix cells.

Table 3 reports both. The within-tier gaps are the expected control: the reference policy has no fitted parameters, so it cannot overfit its train band, and the instrument correctly reads near zero on Calm ($+0.0012$) and noise-signed values on the stressed tiers (the negative Hard and Extreme gaps mean the held-out band happened to be kinder; with no learning, the gap is pure seed noise, and its magnitude, up to 0.34 on Hard, calibrates how much band luck a 16-seed evaluation carries). The per-band bootstrap intervals say the same thing at interval width: Hard's train-band DSR of 0.1231 carries a CI of $[0.000, 0.944]$ and its test band $[0.037, 0.948]$ (the bands are disjoint seed sets, so these are unpaired), overlapping almost entirely, which is what a pure-noise gap looks like.

The transfer matrix (Figure 4) is where regime structure appears, and its cells split by the paper's own noise calibration of 0.34. The Calm-involved point estimates are far above it: source regime Calm scored zero-shot on Hard costs 0.877 of deflated Sharpe, and Calm to Extreme costs 0.973, essentially the whole score. The seed-paired bootstrap intervals temper the two claims differently: Calm to Extreme excludes zero decisively ($[0.635, 0.999]$), while Calm to Hard's interval is wide ($[0.054, 0.999]$), positive throughout but reaching close to zero, so at 16 seeds only the Calm-to-Extreme collapse is resolved beyond doubt. The Hard-to-Extreme cell (0.096, CI $[-0.218, 0.911]$) sits *below* the 0.34 seed-noise calibration and its interval straddles zero, so it is not interpreted: at 16 seeds the instrument cannot distinguish it from band luck. The wide intervals are themselves a calibration result: a bounded, heavily deflated score bootstrapped over 16 seeds is a coarse instrument, and claims narrower than these intervals should not be made from boards this size. The matrix is antisymmetric because the policy is fixed, and its diagonal is zero by construction. Its off-diagonal magnitude is the quantity a within-tier gap is structurally blind to: a policy scored only on calm seeds can carry a saturated DSR into a leaderboard while being worth 0.03 under stress. Both metrics are cheap, and the protocol requires reporting both.

5.5 F4: Stylized-facts realism certification

For each tier, price panels from eight seeded episodes are graded by `certify_realism` against the directional bounds: positive excess kurtosis, positive absolute-return autocorrelation and positive aggregational Gaussianity from the classical catalogue [Cont, 2001], and a super-Poisson Fano factor (the authors' own diagnostic, Section 3.7), with gain/loss skew [Cont, 2001] and timescale asymmetry [Zumbach, 2009] reported as informational.

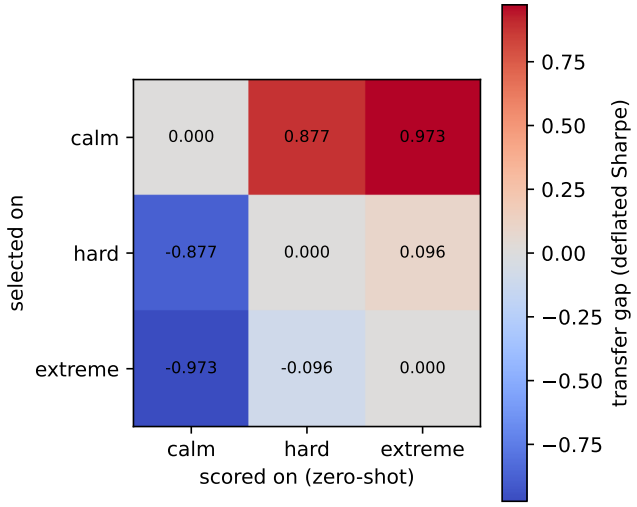


Figure 4: Cross-regime transfer matrix for the reference policy: out-of-distribution deflated Sharpe with the seed band held fixed.

Table 4 and Figure 5 report the certification honestly, and it is a mixed verdict. The certification bounds are universal: they encode what real-market tape looks like, not what each tier intends, so a regime designed to be near-Gaussian is expected to fail them, and does. Calm's returns are platykurtic (mean excess kurtosis -0.97, aggregational Gaussianity -0.58); Calm is therefore, plainly, not realistic tape under this battery, by design, and the gate says so rather than grading it on a curve. Hard and Extreme pass excess kurtosis and aggregational Gaussianity on all eight seeds (means $+16.3$ and $+11.4$ excess kurtosis). But the full conjunction almost never passes: 0/8 on Calm, 0/8 on Hard, 1/8 on Extreme. The binding facts on the stressed tiers are volatility clustering (absolute-return autocorrelation is slightly negative at this 121-bar episode length) and the Fano factor, which drops sub-Poisson exactly when jumps concentrate variance in few bars. This is a finding against the generator, stated as such: the vol-and-jump construction buys fat tails without buying persistent volatility, and a future generator revision must add clustering (for example a stochastic-volatility driver) before the certification gate passes at tier level.

That driver now exists as an opt-in `vol_clustering` knob on the scenario spec: when positive, a deterministic EMA persistence state driven by realized absolute returns modulates each bar's return deviation, using the same mul/add/div/max arithmetic as the tier transforms so cross-runtime byte-identity is preserved, and a committed golden hash pins the clustered output alongside the unclustered tiers. At `vol_clustering` $= 0$ (the default and the canonical leaderboard configuration) the generated panels are byte-identical to the tapes certified above, so the finding stands as reported, and every other experiment in this section runs on this uncertified default tape. Re-certifying the same tiers and seeds at strength 0.5 flips the volatility-clustering fact: the absolute-return autocorrelation pass count rises from 5/8 to 8/8 on Calm, 1/8 to 8/8 on Hard, and 1/8 to 5/8 on Extreme (Hard's mean $|r|$ autocorrelation moves from -0.011 to $+0.029$), lifting Hard's all-facts pass rate from 0.000 to 0.250. Two honesty notes on that remediation. The strength 0.5 was tuned, and re-certified, on the same eight seeds used to diagnose the failure, so the improvement is in-sample on the diagnosis panel; certification on a disjoint seed set at several strengths is the required follow-up before the driver can claim the fact generally. And the remaining binding fact on the stressed tiers is the sub-Poisson Fano factor, which the clustering driver improves (Hard 0.85 to 0.90) but does not yet fix.

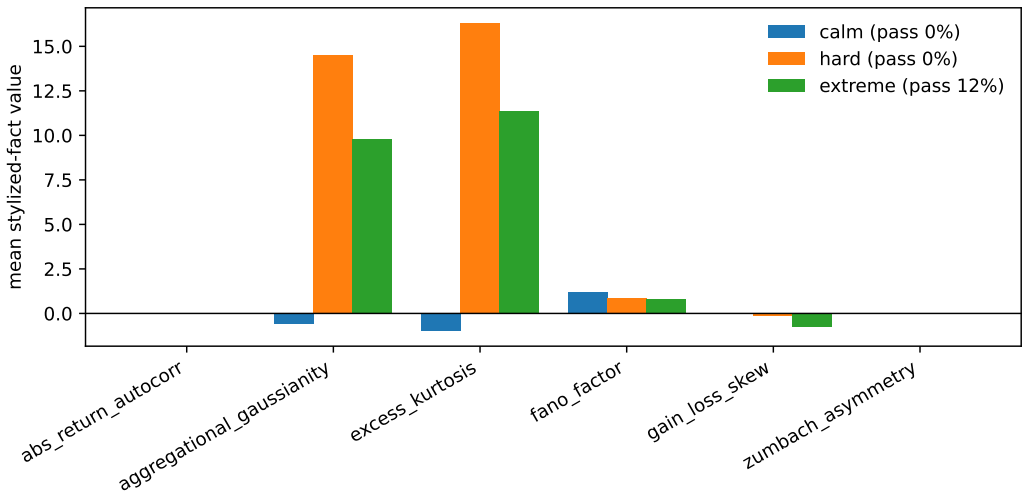


Figure 5: Per-seed stylized-fact values by tier against the certification bounds.

Table 5: Manipulation probe, impact-attributable PnL as fraction of capital per episode, in units of 10^{-4}, mean over 8 seeds with t-based 95% CIs. Left: endpoints of each swept axis (6 grid points per axis); every axis is unprofitable everywhere, so no profitability boundary exists to report. Right: the full size-response grid in the push weight.

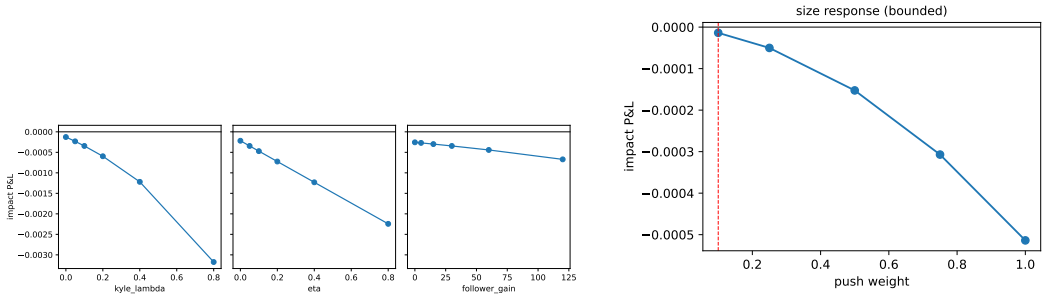
Axis	Low end	High end	Boundary
Kyle λ (0 to 0.8)	-1.3	-31.7 $[-44.3, -19.2]$	none
η (0 to 0.8)	-2.2	-22.4 $[-23.5, -21.4]$	none
Follower gain (0 to 120)	-2.6	-6.7 $[-10.1, -3.3]$	none

Push wt.	PnL	95% CI
0.10	-0.14	$[-0.25, -0.03]$
0.25	-0.50	$[-0.77, -0.24]$
0.50	-1.53	$[-2.05, -1.00]$
0.75	-3.07	$[-3.84, -2.30]$
1.00	-5.14	$[-6.16, -4.12]$

5.6 F5: The manipulation boundary and size response

A push-and-unwind manipulator is scored against its zero-impact paired reference (identical seed and schedule, $\lambda = \eta = 0$) over eight seeds, so `impact_pnl` attributes profit specifically to having moved the price. `impact_boundary_sweep` sweeps permanent impact (Kyle λ), temporary impact (η) and the momentum-follower gain over six grid points per axis; `size_response` sweeps the push weight over five points. The evidence commits per-seed impact-PnL vectors at every grid point; intervals below are t-based 95% CIs over the eight seeds.

One theoretical fact frames the whole finding. The market composes linear permanent impact with temporary impact, and under linear, time-independent permanent impact, round-trip price manipulation is unprofitable essentially by theorem: linearity is exactly the condition that rules out price-manipulation quasi-arbitrage [Huberman and Stanzl, 2004], and the no-dynamic-arbitrage analysis reaches the same conclusion for related impact-and-decay families [Gatheral, 2010]. The clean sweep below is therefore a consistency check, not a certification that manipulation cannot pay in synthetic markets. Section 5.7 runs the concave-impact ablation on the same symmetric schedule and reports a null; Section 5.8 then searches the asymmetric schedules that null did not cover and finds the profit theory predicts, under concave impact only.



(a) Boundary sweeps per impact axis.

(b) Size response in the push weight.

Figure 6: The manipulation probe: impact-attributed PnL is negative everywhere and worsens with impact strength and with size.

The result (Table 5, Figure 6) is the outcome the theory above predicts. On every axis the pump loses money at every sampled coefficient, with every per-point 95% CI excluding zero, including the realistic center of the grid ($\lambda = 0.1$, $\eta = 0.05$: impact PnL -3.4×10^{-4}, CI $[-4.3, -2.6] \times 10^{-4}$). The size response is bounded and decreasing: the least-bad size is the smallest sampled push weight and the loss grows to -5.1×10^{-4} at full size. A profitable region here would contradict the linear specification; the sweep instead agrees with Huberman-Stanzl at every point. The two extensions that follow settle what this sweep cannot. The concave ablation keeps the symmetric schedule and still finds no profit, which is a measurement rather than a corollary; the positive control then shows the same probe finding a profitable asymmetric round trip under concave impact and none under linear, so the linear null above is the theorem confirmed by an instrument that can fail, not one that cannot. The module's disclaimer remains explicit: it diagnoses the simulator, not a trading strategy.

5.7 Concave-impact ablation: making the manipulation probe falsifiable

The F5 result as reported in Section 5.6 carries a structural caveat: under the linear, time-independent permanent impact the market implements, round-trip manipulation is unprofitable essentially by theorem [Huberman and Stanzl, 2004, Gatheral, 2010], so a clean sweep confirms theory but cannot fail. A probe that cannot fail is not yet an instrument. This ablation removes the guarantee. The clearing engine gains an opt-in permanent-impact exponent: the Kyle flow term λQ becomes $\lambda \operatorname{sign}(Q) |Q|^{\kappa}$ in both the temporary fill and the permanent multiplier update, with the Almgren-Chriss own-size term left linear. At $\kappa = 1$ the substitution returns Q itself and the arithmetic is bit-for-bit the published linear path; the exponent is gated onto separate entry points, exactly as the robust-clearing uncertainty set is, because a general power requires `powf`, a libm transcendental excluded from the engine's cross-runtime golden-hash guarantee. All published results and goldens remain pinned to the untouched linear path. Exponents below one give permanent impact concave in flow, the square-root-law regime [Almgren et al., 2005] in which the Huberman-Stanzl argument no longer forbids profitable round trips, so on this arm the probe genuinely can find manipulation, and a null is a measurement rather than a restatement of the specification.

Result: the pump still loses everywhere, but the CIs now say so rather than the theorem. We reran the full F5 protocol at $\kappa = 0.5$ and $\kappa = 0.7$: the same crude push-and-unwind schedule, the same paired zero-impact reference on the same seeds, the same axis and size sweeps, eight seeds per grid point with t-based 95% CIs. No sampled point on any axis is profitable in the mean, no boundary crossing exists, and the size response remains bounded and decreasing on both arms. At the canonical configuration ($\lambda = 0.1$, $\eta = 0.05$, push weight 0.8, follower gain 30) the impact-attributable PnL is -8.9×10^{-4} (CI $[-19.6, +1.8] \times 10^{-4}$) at $\kappa = 0.5$ and -13.5×10^{-4} (CI $[-21.2, -5.9] \times 10^{-4}$) at $\kappa = 0.7$, against -3.4×10^{-4} (CI $[-4.3, -2.6] \times 10^{-4}$) on the linear arm. Losses deepen along the permanent-impact axis on both arms, dramatically so at $\kappa = 0.5$ (mean -0.10 at $\lambda = 0.8$, roughly $32\times$ the linear loss there), and the least-bad push size is again the smallest sampled. Three

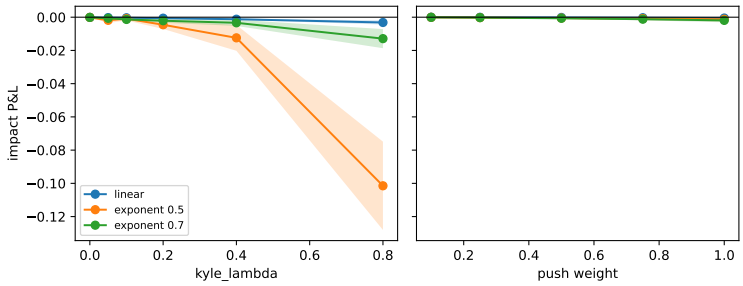


Figure 7: Impact-attributable PnL under linear and concave permanent-impact exponents. The same schedule stays negative, while uncertainty widens on the $\kappa = 0.5$ arm.

of twenty-two concave grid points have CIs that cross zero (the canonical point and the two largest push sizes at $\kappa = 0.5$), so the concave arm is noisier than the linear one, consistent with the amplified price paths it produces, but nowhere does the point estimate turn positive.

Why the null is credible, and what it does not show. Two honest qualifications temper the headline. First, calibration: the exponent applies to raw flow, whose concavity crossover sits at $|Q| = 1$ flow unit, while the probe's per-bar net flow is of order 10^{-3} to 10^{-2}, two to three orders of magnitude below it. On the small-trade side of the crossover a sub-unit exponent raises marginal impact rather than discounting it, so at this calibration the ablation stiffens the book against the pump instead of softening it; that is visible in the deepened losses, and it means the null partly reflects where the probe's flow scale sits, not only the schedule's economics. Second, schedule shape: the Huberman-Stanzl profitable constructions under nonlinear impact exploit asymmetric round trips (accumulate and unwind at different rates), whereas this probe evaluates one fixed, deliberately legible symmetric schedule and one follower-gain range; it does not search over schedules, so the null says this pump does not pay here, not that no schedule can. What the ablation does establish is the property the panel asked for: the instrument now runs in a regime where theory permits it to fire, its answer is an estimate with a confidence interval rather than a corollary, and a future specification change that made the book cheaply manipulable at this flow scale would now surface as a positive region instead of being defined away. This null is the symmetric-arm result. The positive control it calls for, a schedule search over asymmetric round trips at the same calibration, is run in Section 5.8, and it finds the profit the theory predicts at $\kappa = 0.5$.

5.8 Positive control: the probe fires where theory says it should, and only there

The concave-impact ablation of Section 5.7 left one caveat standing: its null was measured on a single symmetric push-and-unwind schedule, whereas the profitable round trips that Huberman and Stanzl [Huberman and Stanzl, 2004] and Gatheral [Gatheral, 2010] exhibit under non-linear permanent impact are asymmetric. Under a concave impact function, splitting a leg of size Q into n equal slices moves the price by $n\,\lambda(Q/n)^{\kappa} = n^{1-\kappa}\lambda Q^{\kappa}$, a factor $n^{1-\kappa}$ more than a single block of the same size, so a manipulator that accumulates in slices and liquidates in a block buys into a price it has pushed up cheaply per unit and sells into a drop it has kept small. Linear impact ($\kappa = 1$) has no such factor and the argument collapses, which is the theorem. A probe that never tries that shape has not been shown able to find it. This fragment closes the gap with a schedule search, and the outcome is a positive control in the strict sense: the instrument fires under the conditions where theory permits manipulation and stays silent under the conditions where theory forbids it.

The asymmetric family and the search. `AsymmetricSchedule` in `sharpearena.manipulation` parameterizes a flat-to-flat round trip by the durations of its two legs (`up_bars`, `down_bars`) and a block fraction (`size_split`): the share of each leg's notional executed in a single bar at the turn of the trip, the remainder spread uniformly over the leg's other bars, with the uniform ramp recovered at `size_split` $= 1/n$. Equal uniform legs

reproduce the symmetric schedule bar for bar, and the attribution is unchanged: every run is paired with its own zero-impact reference on the same seed. The search sweeps five duration ratios (1:9, 2:8, 5:5, 8:2, 9:1) over the symmetric trip's ten trading bars, three block fractions (uniform, 0.5, 0.9), and exponents $\kappa \in \{1.0, 0.7, 0.5\}$, on three arms: a theory arm with no followers and no temporary impact ($\eta = 0$, gain 0), a temporary-impact arm ($\eta = 0.05$, gain 0), and the canonical follower ecology of the published arms ($\eta = 0.05$, gain 30). Every point carries eight seeds and a t-based 95% CI; 135 grid points in all. The question asked of the grid is whether any round trip is profitable in the mean.

Result: profit appears under concavity with asymmetry, and nowhere under linear impact. On the theory arm at $\kappa = 0.5$ the slow-accumulate, block-liquidate trip (nine bars up, one bar down, uniform) earns an impact-attributable $+21.8 \times 10^{-4}$ (CI $[+17.4, +26.2] \times 10^{-4}$), with the neighbouring 8:2 trip at $+3.2 \times 10^{-4}$ (CI $[-0.4, +6.7] \times 10^{-4}$); the mirror trip (1:9, block-accumulate, slow-liquidate) loses -118.9×10^{-4}, and the symmetric 5:5 trip loses -32.7×10^{-4}. (On the canonical arm the uniform 5:5 point reproduces the ablation's symmetric number exactly, -8.9×10^{-4} with CI $[-19.6, +1.8] \times 10^{-4}$, confirming that the family contains the published schedule as a member.) Adding temporary impact ($\eta = 0.05$) trims the winner to $+18.4 \times 10^{-4}$ (CI $[+13.9, +22.8] \times 10^{-4}$) without removing it. At $\kappa = 0.7$ no point is profitable, the least-bad trip being again 9:1 at -4.3×10^{-4} (CI $[-5.4, -3.1] \times 10^{-4}$), against -29.2×10^{-4} for its mirror: the ordering theory predicts is already present, but the $n^{1-\kappa}$ factor at $n = 9$ (1.93 at $\kappa = 0.7$ versus 3.0 at $\kappa = 0.5$) is not yet large enough to overcome the temporary and own-size costs at this flow scale. At $\kappa = 1$ every one of the 45 points on all three arms is negative, with the best at -1.3×10^{-4}, and the two mirror trips are indistinguishable (-3.58 versus -3.58×10^{-4} on the theory arm), which is the time-reversal symmetry of linear impact showing up in the numbers. Concentrating a block at the turn destroys the winner: at $\kappa = 0.5$ the 9:1 trip falls to -16.3×10^{-4} with a block fraction of 0.5 and to -62.9×10^{-4} at 0.9, because the block un-slices the accumulation leg, and slicing is the entire mechanism. In the canonical follower ecology no point is profitable at any exponent (the 9:1 trip at $\kappa = 0.5$ reads -32.8×10^{-4} with CI $[-89.6, +23.9] \times 10^{-4}$): the momentum followers chase the slow accumulation, and their flow moves the book against the manipulator's block. Figure 8 shows the full grid.

What this establishes and what it does not. The probe now has both halves of a calibration. It returns a positive with a CI clear of zero exactly where Huberman-Stanzl and Gatheral say a positive must exist (concave permanent impact, sliced accumulation, block liquidation, no crowd), it returns the theory's ordering at the intermediate exponent, and it returns a uniform null under linear impact, where the theorem says the null is the truth. The F5 headline therefore stands as a measurement of the shipped specification rather than a limitation of the instrument: a symmetric pump does not pay under linear impact, and an asymmetric one does not pay there either. The search did not cover the push size (fixed at the canonical 0.8), the flow calibration ($\lambda = 0.1$ and the sub-unit flow scale of Section 5.7, unchanged), legs longer than ten bars or holds longer than one, overshooting or short-side round trips, or follower gains other than 0 and 30; a profit in any of those under linear impact would contradict the theorem and would be a bug report against the engine, not a finding.

5.9 F6: Adverse-selection markout decomposition

`compare_informed_vs_uninformed` runs twenty-four paired episodes in which the identical meta-order flow is sided on the alpha in one leg and on a coin flip in the other, holding schedule, sizes and price path fixed, and reports mean maker markout per filled unit at horizons of 1, 5 and 20 bars. Markout is measured in the tape's absolute price units (the mid starts at 100) per unit of filled quantity; the evidence commits per-episode vectors, and the gap CIs below are t-based 95% over the twenty-four paired episodes.

Two certifications must be separated here: the sign of the *gap* and the sign of the *levels*, and they come out on opposite sides.

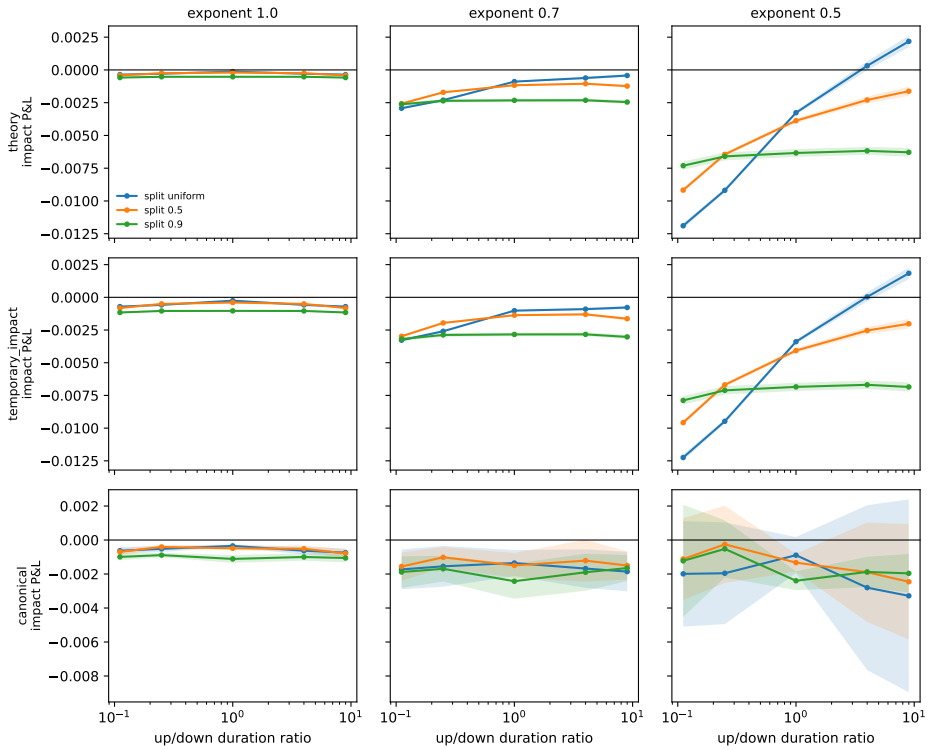


Figure 8: Positive control for the manipulation probe. Impact-attributable P&L of asymmetric round trips against the up/down duration ratio (log scale), one column per permanent-impact exponent, one row per arm, one line per block fraction, shaded 95% CIs over eight seeds. The acceptance is a single region: slow-accumulate, block-liquidate, uniform slicing, $\kappa = 0.5$, no followers.

Table 6: Maker markout per filled unit (price units per unit quantity) against informed versus paired-uninformed flow, 24 paired episodes; gap CIs are t-based 95% over per-episode paired gaps. The gap is the price of trading against information and grows with horizon; the levels stay positive at every horizon, a separate finding discussed in the text.

Horizon (bars)	Informed	Uninformed	Gap	Gap 95% CI
1	0.689	0.761	0.072	$[0.052, 0.094]$
5	0.489	0.791	0.302	$[0.228, 0.383]$
20	0.386	0.823	0.436	$[0.303, 0.579]$

The gap certification passes at every horizon (Table 6, Figure 9): makers filled by informed flow mark out worse than the paired-uninformed control at $h = 1$, 5 and 20, the per-unit gap widens from 0.072 to 0.436 as the information realizes, and every horizon's paired CI excludes zero, including the smallest ($h = 1$: $[0.052, 0.094]$ over per-episode paired gaps). That is the signature shape of adverse selection: the loss is in the drift after the fill, not in the fill itself. The claim is scoped to the gap, the differential cost of informed over uninformed flow, and not to maker profitability levels.

The level result goes against the environment, and we report it with the same candor as F4. Makers earn a *positive* mean markout against informed flow at every horizon (0.689, 0.489, 0.386 per unit), which is on the wrong side of the Section 2 standard that a realistic market should not let makers profit from informed flow. The cause is calibration and design, not accounting: fills are struck at the pre-move mid, so every fill books the full quoted depth as spread capture, the default quote depths are wide relative to the drift the $\alpha = 2.0$ meta-orders realize over 20 bars, and the price path is exogenous, so quotes never widen in equilibrium response to toxicity as they would in Glosten and Milgrom [1985]. At this calibration the probe therefore measures markout accounting against a fixed

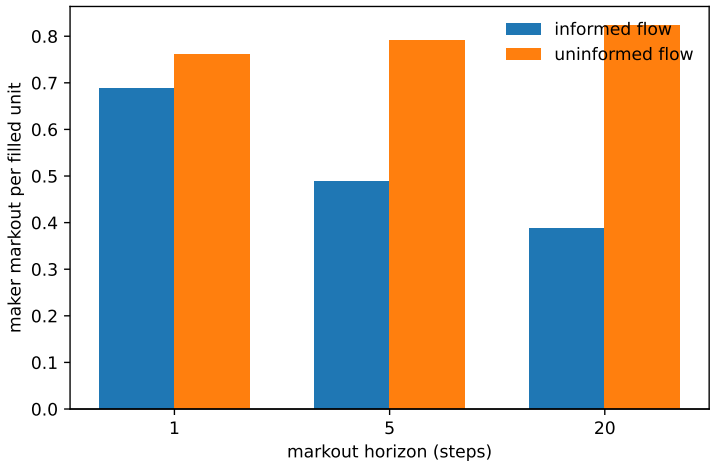


Figure 9: Markout per filled unit by horizon: informed flow versus the paired-uninformed control.

Table 7: Failure-mode counts per tier, 128 episodes each (8 policies $\times$ 16 seeds). No episode reaches bankruptcy or a cascade wipe; failure is dominated by mandate breaches, and drawdown breaches triple from Calm to Extreme.

Tier	Clean	Stopped out	Mandate struct.	Mandate DD	Mandate inv.	Bankrupt	Clean rate
Calm	72	0	36	11	9	0	0.56
Hard	68	0	36	14	10	0	0.53
Extreme	50	5	31	32	10	0	0.39

drift, in which informed flow costs makers relative to uninformed flow but not in absolute terms. Recalibrating the informed alpha or striking fills at post-impact prices so the level turns negative, and adding a variant where quotes widen with realized toxicity, are the named follow-ups.

The single-episode maker decomposition illustrates the exact identity on one seeded episode (seed 0); it is an illustration, not an aggregate finding. The wider quoter (`maker_0`, 135 fills) keeps a positive markout at $h = 20$ (0.526 per unit, toxic-fill rate 0.296): the flow is not selecting against it. The tighter quoter (`maker_1`, 92 fills) shows the subsidized pattern: aggregate markout is still positive at $h = 20$ (0.197 per unit) but its meta-order fills alone lose 0.197 per unit on 62% of its filled quantity, with a toxic-fill rate of 0.533 at $h = 20$, so noise flow is paying for the informed flow. Spread capture and adverse drift decompose exactly (`spread_capture` + `adverse_drift` = `markout`, in price units times quantity): `maker_1` captures 82.8 of spread and gives back 64.7 to adverse drift by $h = 20$.

5.10 F7: Failure-mode distributions

Every rollout of the eight-policy field (the six reference policies plus the two behavioral counterparties), across all three tiers and sixteen seeds (384 episodes), is classified by `classify_episode_failure` into the worst-case-first taxonomy: cascade-wiped, bankrupt, stopped-out, mandate breach (structural, drawdown or inventory), or clean, with a per-scenario sampled mandate and a 50% drawdown stop in force.

One mechanism drives the largest counts and must be stated first: mandates are sampled per scenario and assigned to fixed policies that cannot observe or adapt to them, so a structural breach is guaranteed whenever the draw pairs, say, a shorting policy with a long-only mandate. The flat structural-breach counts across tiers (36, 36, 31) are the signature of that assignment mechanism, not a behavioral finding about agents; a mandate-aware field could invert the distribution entirely, and measuring one is the interesting follow-up.

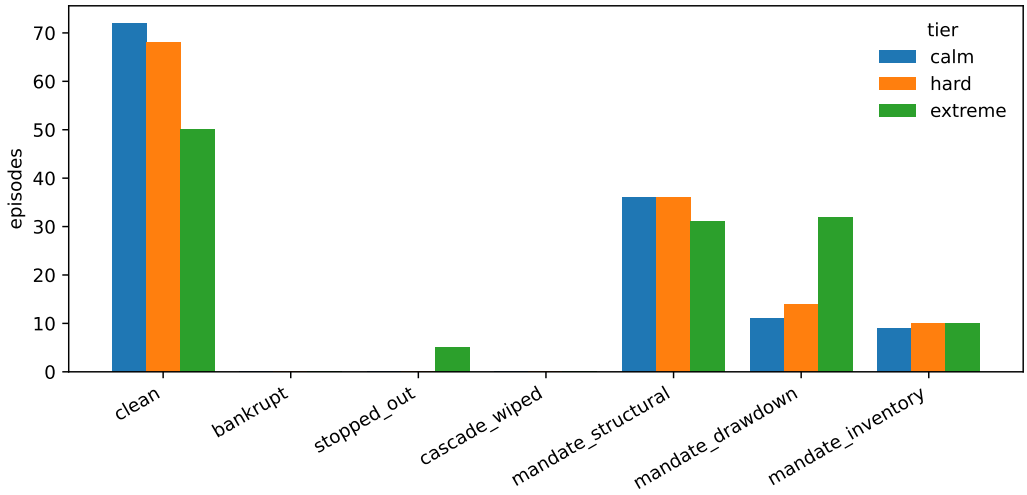


Figure 10: Failure-mode distribution per tier per policy over 384 classified episodes.

Table 7 and Figure 10 show the shape of failure as stress rises. The clean rate falls from 0.56 on Calm to 0.39 on Extreme. Structural mandate breaches are roughly flat across tiers, as the mechanism above dictates: a shorting policy under a sampled long-only mandate breaks the rule regardless of the weather. The stress-sensitive modes are the drawdown family: mandate drawdown breaches triple from 11 on Calm to 32 on Extreme, and hard stop-outs at the 50% line appear only on Extreme (5 episodes, four of them `max_sharpe`). No policy in this field ever reaches bankruptcy or a cascade wipe; the taxonomy's worst states are reachable but not by these baselines at this stop level. Per policy, `flat` is clean 48/48 by construction, while `overconfident` is the worst citizen on Calm (7 of its 16 episodes end in a drawdown breach) and `max_sharpe` is the worst on Extreme (3 clean of 16). The headline, scoped to what was measured: on this mandate-blind reference field, half of everything that goes wrong is process, not PnL, which is exactly the half a return-ranked board never sees. Whether the same holds for agents that can read their mandate is an open measurement, not a claim of this paper.

5.11 F8: Ecology outcomes under shocks, replicated over seeds

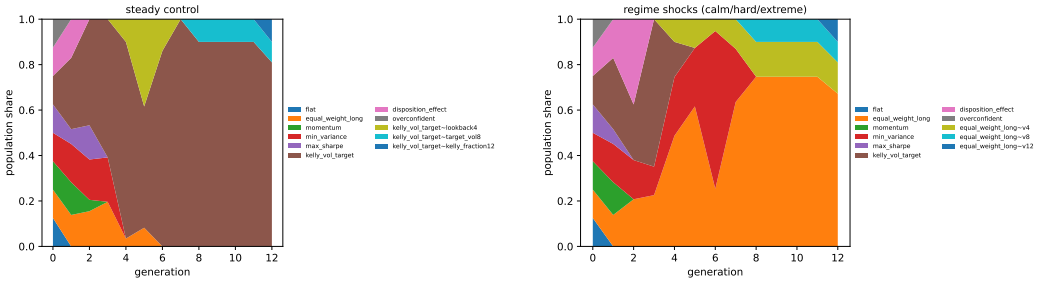
`run_ecology` seeds the replicator with the eight baseline species over a shared-book market via `market_payoffs` (12 generations, field size 8, innovation every 4 generations breeding parameter-perturbed variants of the leader) and runs trajectories under a steady all-Calm control schedule and a `regime_shocks` schedule rotating Calm (generations 0 to 3), Hard (4 to 7) and Extreme (8 to 11). The probe is replicated over eight replicator seeds per schedule; Table 8 reports the cross-seed winner distribution, and the seed-0 trajectories (Figure 11) remain the illustrative detail. Note the two schedules chain episode seeds differently (the shock schedule strides seeds across generations), so control and shocked runs at the same replicator seed are matched in configuration, not in price paths.

The replicator flow rule sits in the market-ecology tradition: strategies as species whose profitability decays as their share of the flow grows [Farmer, 2002], and populations whose composition, not any single agent's optimization, drives aggregate dynamics [Lux and Marchesi, 1999]. The implementation abstracts away much of what those models study: capital flows react instantly (no fund-flow lags), there are no leverage or margin constraints beyond the strategies' own sizing, and there is no entry beyond the single leader-perturbation innovator. Its outputs are trajectories of this replicator map, not claims about market ecology in general.

The replication overturns the single-seed narrative, and we report that plainly. At seed 0 the story was clean: `kelly_vol_target` sweeps the control board (peak share 1.00, final 0.81), collapses under

Table 8: Replicated ecology outcomes over 8 replicator seeds per schedule: distribution of the final dominant lineage (bred variants credited to their founding species). The root winner differs between the schedules on 5 of 8 seeds; the seed-0 replacement of the volatility targeter by the unlevered long book occurs on 1 of 8 seeds.

Final dominant lineage	Control (all Calm)	Shocked (Calm/Hard/Extreme)
kelly_vol_target	4	7
equal_weight_long	2	1
flat (no resolved dominant)	2	0



(a) Control: steady Calm.

(b) Shocked: Calm, then Hard, then Extreme.

Figure 11: Seed-0 population-share trajectories over 12 replicator generations, the illustrative single run behind Table 8's distribution. At this seed the regime rotation replaces the calm-regime winner with the unlevered long book; across the eight-seed replication that replacement occurs once.

the shock schedule once the regime turns Hard, and `equal_weight_long` inherits dominance (final 0.67). Across eight seeds that story is the exception. Under the shock schedule a `kelly_vol_target` lineage ends dominant on 7 of 8 seeds (final shares near 0.90); the seed-0 handoff to the long book happens once. The control side is itself seed-dependent: `kelly_vol_target` wins 4 seeds, `equal_weight_long` wins 2, and on 2 seeds no species resolves dominance at all (all eleven end persistent at small shares, with a `flat` variant holding the largest share at 0.10). The root-level winner differs between the two schedules on 5 of 8 seeds, so shocks do reorder outcomes more often than not, but with no consistent direction. Extinction pressure is more stable than identity: shocked runs produce a resolved dominant on 8/8 seeds with 4 to 7 extinctions per run (at seed 0, both behavioral counterparties are gone or collapsed on both schedules, `overconfident` by generation 1).

The honest conclusion is methodological. One replicator trajectory is one draw from a heavy-tailed outcome distribution, and single-run ecology narratives, including the one this paper's earlier draft promoted to the abstract, do not survive replication. What the probe reliably produces at this scale is the outcome distribution itself: which lineages can win, how often shocks reorder the board, and how much of the population goes extinct, per committed seed set. Sharper claims need more seeds, varied shock orderings and path-matched schedule pairs, all of which are mechanical extensions of the committed script.

5.12 Predictability probe: measuring the in-band inversion attack

Section 7 names a threat the leak-free interface property does not cover: a scenario is a pure function of a public 64-bit seed under a public deterministic transform, so future bars are in principle computable from past bars, and no forbidden operation is needed. This probe converts that named threat into a measurement. Three adversaries predict the next bar's return from a causal prefix only, on the three volatility tiers, 16 seeds per tier (4 symbols, 120 days, 30 warmup bars): an *unconditional baseline* that predicts the expanding prefix-mean return per symbol; an *honest statistical adversary*, a ridge AR(5) refit on the prefix at every bar, with no knowledge of the generator; and a *known-seed oracle*, which replays the generator from the true seed and therefore predicts every future bar exactly, the upper bound on any inversion attack. Each predictor is also run as a frictionless unit-gross

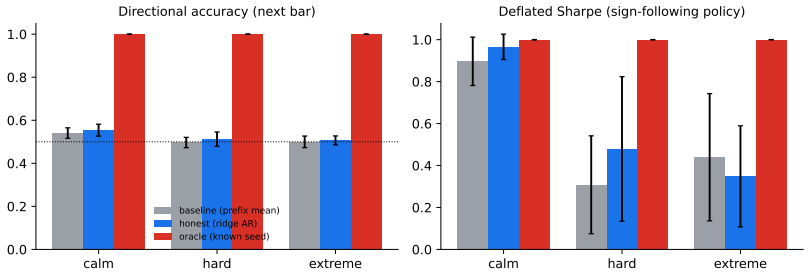


Figure 12: Next-bar directional accuracy by tier. The causal AR adversary remains near the unconditional baseline; knowing or recovering the seed gives perfect prediction.

long/short sign-following policy and scored through the public `score_run` kernel; we report the kernel's deflated-Sharpe probability.

The honest adversary extracts almost nothing. On Calm the ridge AR reaches $55.4\% \pm 2.8\%$ directional accuracy against the baseline's $54.1\% \pm 2.4\%$: it detects the tier's designed momentum autocorrelation, worth about one accuracy point over drift-following, and its policy DSR (0.97 ± 0.06 vs. 0.90 ± 0.12) reflects the same mild, real, and disclosed structure rather than any transform inversion. On the stressed tiers the edge vanishes: $51.2\% \pm 3.3\%$ on Hard and $50.6\% \pm 2.1\%$ on Extreme, statistically indistinguishable from coin flipping, with MSE no better than the unconditional baseline (Hard: 1.24×10^{-4} vs. 1.22×10^{-4}; Extreme: 1.27×10^{-3} vs. 1.17×10^{-3}) and policy DSRs of 0.48 ± 0.35 and 0.35 ± 0.24. The amplified tiers behave as near-martingales to a regression adversary: in-band statistical prediction recovers the disclosed stylized structure and nothing beyond it.

The oracle bound is total, and a bounded seed search reaches it. The known-seed oracle scores 100% accuracy, zero MSE, and DSR 1.00 on every tier: determinism means the gap between an honest adversary and a successful inverter is the entire distance from coin flipping to omniscience, so the question is only the cost of inversion. We implemented the strongest cheap version: a brute scan over a bounded seed band of width 2^{16} centered on the published `start_level`, matching each candidate's first-bar closes against the single observed opening bar. The scan recovered the true seed in 16 of 16 trials with zero collisions, from a prefix of one bar, at $14.7\,\mu$s per candidate (≈ 0.96 s per scan); every recovered seed was then verified to reproduce the full 30-bar observed prefix on its tier. The lesson is sharp: when the deployed seed band is bounded and public, as a `(start_level, num_levels)` spec makes it, in-band inversion is not cryptanalysis but a table scan, and any such deployment should treat its scenarios as fully known to a motivated adversary. Extrapolating the measured per-candidate cost to the unbounded 2^{64} space gives ~ 8.6 million CPU-years, so brute-force inversion of an unpublished seed drawn from the full space is infeasible by enumeration.

What remains open. The probe brackets the attack but does not close it. Between the honest regression (no edge beyond disclosed structure) and the band scan (total recovery given a bounded public band) lies the case that matters for unbounded deployments: analytic inversion of SplitMix64 through the mul/add/div/max price transform from observed reals. SplitMix64 is not a cryptographic generator; its state transition is a fixed odd-constant addition and its output mixing is invertible, so recovering the state from a handful of *exact* raw outputs is straightforward. What the adversary observes here is not raw outputs but prices, each a truncated 53-bit draw passed through data-dependent arithmetic with floors, and whether that quantization and composition resists lattice or guess-and-determine state recovery is a cryptanalytic question this paper does not answer. The honest reading of the probe: the environment's unpredictability rests on the secrecy and entropy of the seed, not on any hardness of the transform, and published bounded seed bands forfeit both.

5.13 Sealed evaluation seeds: closing the measured hole

The predictability probe (Section 5.12) left one attack fully realized: when the deployed seed band is bounded and public, in-band inversion is a table scan, not cryptanalysis. The shipped held-out set is such a band. Its named seeds are public constants, `EVAL_SEED_BASE` plus fixed offsets, so anyone with the repository can enumerate them, replay every held-out scenario in advance, and present a policy that has memorized the answers. The disjointness proof (every eval seed lies at or above `EVAL_SEED_BASE`, every train seed below it) says nothing about this; it guarantees the agent never *trained* on an eval seed, not that the eval seeds are unknown. This section adds the opt-in mechanism that closes exactly that hole, and measures the closure with the probe's own adversary.

Mechanism. A sealed evaluation keeps the public band *structure* and hides the concrete seeds behind a salt. The evaluator supplies a secret salt k (at least 16 random bytes; the derivation is only as unguessable as the salt), and the seed for named slot i is

$$\text{seed}_i \;=\; \texttt{EVAL\_SEED\_BASE} \;+\; \big(F(k,i) \bmod (2^{64} - \texttt{EVAL\_SEED\_BASE})\big),$$

where F absorbs the salt bytes and their length with FNV-1a/64 and drives the digest, a fixed domain tag, and the slot index through three rounds of the SplitMix64 finalizer, chaining the digest back in between rounds. It is built entirely from primitives the crate already carries (no cryptographic dependency was added), and it is exposed identically from Rust (`sealed_seed(salt, slot)`), the pyo3 binding, and Python (`sealed_eval_seeds(salt)` returns the same `name`→`seed` map as the public set; `evaluate_eval_set(..., salt=)` runs the unchanged protocol on it). Three properties are pinned by tests on both sides of the binding: every sealed seed lands in $[\texttt{EVAL\_SEED\_BASE}, 2^{64})$, so disjointness from the train band holds by construction and is checkable by anyone *without* the salt; the map is deterministic in (k, i), with a cross-language pinned value; and distinct salts (down to a single flipped bit) give distinct seeds. The public set, its version string, and the committed regression snapshot are untouched; sealing is opt-in.

We are explicit about what the construction is. It is a keyed PRF-*style* derivation, not a certified MAC: FNV-1a is not collision resistant and the SplitMix64 finalizer is a public bijection, so an adversary holding several revealed (i, seed_i) pairs may be able to recover the absorbed salt digest algebraically. What it buys is the property the probe showed was missing: with a high-entropy salt, an adversary who knows the band structure, the slot names, the derivation, and the generator cannot enumerate the seeds, because the 2^{64}-wide band is no longer collapsed onto a 2^{16}-wide public window. What it does not buy is any hardness of the generator itself: once a sealed scenario's tape is observed, the adversary faces the same open question as before, analytic inversion of SplitMix64 through the price transform, and a revealed salt spends every seed it derives.

Commit-reveal workflow. The salt lives outside the repository and outside the evaluated agent's reach. Before the run the evaluator publishes a commitment, $\text{SHA-256}(k)$, which pairs naturally with SharpeBench's forward attestation: the attestation binds the commitment before any score exists. After the run the salt is revealed; anyone can recompute the seeds from (k, i), check the commitment, and replay the evaluation byte for byte through the same deterministic generator and kernel. The evaluation is therefore secret while it matters and reproducible afterwards, and a salt is single use.

The probe's adversary, re-run. We re-ran the band-scan adversary of Section 5.12 unchanged against two derivations of 16 named slots: *public*, the eight committed offsets plus eight further offsets drawn uniformly from the same 2^{16} band (a hypothetical extension of the public set, derived the same way); and *sealed*, the same 16 slots under a 32-byte salt from the operating system's entropy source, committed by hash before the run. The adversary builds the first-bar-close table for the 2^{16} candidates at the public band start (0.75 s, about 11 μs per candidate), matches the single observed opening bar against it (a few milliseconds per trial), and verifies any hit against the 30-bar prefix on the Hard tier. On the public band it recovered 16 of 16 seeds with zero collisions, all 16 prefix-verified: the shipped held-out set is fully known to this adversary. On the sealed band it recovered 0 of 16; the pair the mechanism is judged by is 16/16 public against 0/16 sealed. That is not luck at the margin: the scan

covers a $2^{16}/(2^{64} - 10^6) \approx 3.6 \times 10^{-15}$ fraction of the band, so the expected number of recoveries at this budget is 5.7×10^{-14}, and the probe's own extrapolation (~ 8.6 million CPU-years for the full space) is the cost of extending the scan until it succeeds. Finally, revealing the salt replayed all 16 sealed scenarios exactly (16 of 16 first-bar matches against the recomputed seeds), which is the commit-reveal guarantee in operation. Evidence, including the commitment, the revealed salt, and every per-trial record, is in `paper/evidence/sealed-seeds.json`.

Scope. This closes the specific hole the probe measured, the public bounded band, and nothing more. The honest statistical adversary of Section 5.12 is unaffected in either direction (it never used the seed), and the open cryptanalytic question about SplitMix64 through the price transform remains open. The environment's unpredictability still rests on the secrecy and entropy of the seed; sealed seeds are the mechanism that lets an evaluator actually supply both, while keeping the disjointness proof public and the evaluation replayable.

6 Related work

Environment interfaces. Gym [Brockman et al., 2016] demonstrated that a small, stable environment interface is worth more than any single environment, and Gymnasium [Towers et al., 2024] carried the interface forward with versioned environment IDs and a conformance checker; SharpeArena's Python surface is a first-class Gymnasium citizen with versioned, namespaced IDs and adopts its `check_env`. PettingZoo [Terry et al., 2021] standardized the multi-agent case, which SharpeArena's shared-book and batched-competition environments implement. Minari [Farama Foundation, 2024] standardizes offline-RL datasets; SharpeArena exports leak-safe rollouts to it. The wire contract of Section 4 differs from all of these in scope: it is a language-agnostic JSON boundary for external agent processes, governed like a protocol rather than a library API.

Evaluation rigor in RL. The environment's evaluation stance imports three lessons from the RL evaluation literature. From the ALE revisitation [Machado et al., 2018]: determinism in the environment invites trajectory memorization, so evaluation must be designed against it; SharpeArena keeps determinism for verifiability and defeats memorization with procedural scenario distributions and held-out seed bands instead of injected stochasticity. From Procgen [Cobbe et al., 2020]: procedural generation over integer seed intervals makes generalization measurable, which Section 3.4 ports to markets. Prioritized Level Replay [Jiang et al., 2021] shows seed-level curricula shape what agents learn, which motivates keeping the train band operator-owned and the held-out band frozen. τ-bench [Yao et al., 2024] argues that agent reliability must be measured across repeated runs rather than on average; that requirement lives in the SharpeBench kernel this environment feeds [Toca, 2026].

Financial RL environments. A Gym-for-finance lineage predates this work, and SharpeArena should be read against it on the four property axes it competes on: leak-freedom mechanism, determinism guarantee, trajectory verifiability, and the external-agent contract, with the generalization protocol as a fifth. FinRL [Liu et al., 2020] and FinRL-Meta [Liu et al., 2022] wrap historical market data into gym-style training environments at scale; FinRL-Meta's DataOps pipeline explicitly targets the data-quality half of the problem (survivorship bias, backtest overfitting). Because their environments replay recorded data inside a general-purpose Python process, leak-freedom is a matter of pipeline discipline rather than interface shape, and neither system claims byte-determinism across platforms or third-party replay verification of submitted trajectories; their agent surface is the in-process Gym API rather than a language-agnostic wire contract. ABIDES-Gym [Amrouni et al., 2021] wraps the ABIDES discrete-event market simulator [Byrd et al., 2020] in Gym environments, bringing multi-agent market microstructure to RL training; its focus is simulation fidelity, and to our knowledge it does not target cross-runtime byte-determinism or tamper-evident trajectories. TradeMaster [Sun et al., 2023] is a holistic platform spanning data, environments, algorithms and evaluation for RL trading research; it standardizes the workflow rather than hardening the producer, and its evaluation toolkit runs inside the same trust domain as the agent. mbt_gym [Jerome et al.,

2023] provides model-based Gym environments for limit-order-book problems, including the same Avellaneda-Stoikov family as Section 3.5, with closed-form solutions available as baselines; it is the closest relative of the market-making task specifically, and Section 3.5 notes the relation. Where these descriptions rest on our reading of the systems' papers and repositories rather than on properties their authors claim, we have kept them descriptive; none of these systems, to our knowledge, sets out to make look-ahead structurally unavailable at the interface, pin cross-runtime byte-identity with committed hashes, or make submitted trajectories third-party recomputable, which is the specific combination this paper contributes.

Market simulation. ABIDES [Byrd et al., 2020] is the closest simulator in spirit: a high-fidelity multi-agent market with a matching engine and background agent populations. SharpeArena trades some of that fidelity for properties ABIDES does not target: cross-runtime byte-determinism with committed golden hashes, a structurally leak-free observation boundary, replay-based tamper evidence, and a governed external-agent wire contract. The microstructure models are the classical ones: Kyle's permanent impact [Kyle, 1985], Almgren-Chriss temporary impact and execution cost [Almgren and Chriss, 2001], and the Avellaneda-Stoikov quoting model with its closed-form optimum [Avellaneda and Stoikov, 2008]. The realism gate certifies the generator against Cont's stylized facts [Cont, 2001].

Scoring. The environment deliberately contains no scorer. The deflated Sharpe [Bailey and de Prado, 2014] and the reliability and process gates that consume this environment's trajectories are specified in the SharpeBench paper [Toca, 2026]; this paper is the producer half of that funnel.

7 Limitations

One synthetic generator family. All three tiers come from a single procedural vol-and-jump generator; no real price series enters the environment, and the tiers differ in that generator's parameters, not in kind. F4 quantifies the cost: the tape has fat tails on the stressed tiers but no volatility clustering at episode length, and the full stylized-facts conjunction passes on only 1 of 24 certified runs. An agent that excels here has demonstrated skill against this generator's structure, not against a market, and the realism gate says so rather than hiding it. This is why the abstract and Section 1 frame every finding as instrument calibration on uncertified tape; the framing is a constraint on the claims, not a footnote to them.

The interface guarantee is not an information guarantee. The leak-freedom of Section 3.1 is a statement about the interface's shape: no operation returns a post-cursor bar. It is not a claim that public deterministic scenarios are unpredictable. Section 5.12 makes the distinction empirical. A causal AR(5) adversary recovers only the disclosed Calm momentum and stays near chance on Hard and Extreme, but an exhaustive scan of a public 2^{16} seed band recovers all 16 tested seeds from one observed bar in about one second. Extrapolation makes brute force over the full 64-bit space infeasible, yet analytic inversion through quantized SplitMix64 outputs remains untested. Seed bands defend against memorized episode identity only when the held-out seeds retain entropy and secrecy; bounded public evaluation bands should be treated as known scenarios. The deployed mitigation is the sealed-seed mechanism of Section 5.13: an opt-in keyed derivation into the held-out band under a commit-reveal salt, against which the same scan recovers 0 of 16 seeds and the revealed salt replays 16 of 16. That closes the bounded-band hole specifically. The derivation is PRF-style, not a certified MAC, its security is the salt's entropy and secrecy, and the cryptanalytic question, whether an observed tape lets an adversary invert SplitMix64 through the price transform, stays open. "Leak-free" therefore means exactly the interface property and no more.

The baselines are trivial policies, not trained agents. Every number in Section 5 is produced by fixed reference policies (buy-and-hold analogs, one-step tilts, closed-form allocators) and two deliberately biased behavioral counterparties. They establish the honest zero and calibrate the instruments; they do not establish what a trained agent scores, and this paper's claims are scoped to

environment and protocol for that reason, with compute as the stated constraint. In particular, the near-zero within-tier generalization gaps in F3 are the expected control for policies with nothing to overfit, not evidence that the environment resists overfitting by learners. Relatedly, no reference policy is rank-eligible under the kernel's full conjunction (Section 5.1). Section 5.2 establishes that the eligibility set is non-empty on every tier, but it does so with an out-of-band oracle that replays the seed one bar ahead, which is not an entrant and is not scored on any board; no policy that reads only the observation interface has yet been shown eligible.

Named future experiments. Two follow-ups remain specific. (1) A learning baseline: PPO trained through the shipped Gymnasium adapter on the train band, evaluated held-out and cross-regime, so F3 contains a policy that can actually overfit. (2) Analytic state recovery from quantized SplitMix64-derived prices, closing the gap between the bounded-band scan and full-space brute force; sealed seeds (Section 5.13) remove the bounded-band attack but do not answer this one. Two named earlier are done. The manipulation positive control left by Section 5.7 is run in Section 5.8: an asymmetric schedule search finds a profitable round trip at $\kappa = 0.5$ and none under linear impact, though the search did not cover push size, λ, legs longer than ten bars, holds longer than one bar or short-side trips. The rank-eligibility existence question of Section 5.1 is settled by the witness of Section 5.2, by an oracle rather than by an entrant.

The Python probe layer is outside the golden-hash guarantee. The byte-identical claim covers the Rust core and its WebAssembly and Python bindings, where golden hashes and parity tests enforce it. The Python reductions that produce the realism, manipulation, adverse-selection, failure, ecology and predictability evidence are deterministic in their seeds, but are off the golden-hash path; their floating-point reproducibility is the platform's, not the paper's. The concave-impact engine itself is also excluded from the canonical byte-identity claim because a general exponent uses `powf`; the linear default remains on the untouched golden path.

The impact models are stylized. The endogenous market composes Kyle and Almgren-Chriss terms; the adverse-selection module does not separate a meta-order's information from its impact; the follower population is a one-parameter caricature. The concave ablation is not a universal test: its raw flow sits two to three orders of magnitude below the $|Q| = 1$ crossover, so exponents below one amplify impact at the tested calibration, and the symmetric schedule does not search the asymmetric round trips nonlinear theory permits. Section 5.8 does search them and finds one that pays at $\kappa = 0.5$ with no followers, so the symmetric null says this pump does not pay here, and the positive control says the instrument can see one that does; what neither covers is push size, λ, longer legs, longer holds or short-side trips.

The guard is a deny-list. The structural guarantee covers the environment's own surface. `LookaheadGuard` extends it to harness tools by pattern, and a deny-list is exactly as good as its patterns; a novel leak path through a side channel the environment does not own, wall-clock time, filesystem state, a network call, is out of scope for both layers. The replay check verifies decisions against frozen data and therefore catches falsified results, not information an agent obtained elsewhere.

No live external entrants yet. The contract, the conformance kit and the replay verifier exist, and the evidence run exercises them, but no third party has yet submitted an agent. A hosted intake, a live leaderboard and multi-tenant isolation of untrusted agent processes do not exist, and the governance document is a written promise backed by tests that has not been stressed by a hostile or careless external implementer. The significance claim of this paper is therefore the verifiability infrastructure that is checkable today; adoption is future work, not an assumption.

Broader impact. Two dual-use surfaces deserve an explicit norm. First, leaderboard scores: a byte-verifiable score is more persuasive, not more externally valid, and a verified SharpeArena number is a claim about a synthetic environment only. SharpeArena scores are not performance

marketing, and presenting them to retail audiences as evidence of real-market skill would be a misuse the project's documentation states as out of bounds. Second, the manipulation probe: it is a payoff-search harness over manipulation schedules, which is dual-use by construction. We judge publishing it net-positive because, under the linear-impact specification it ships with, theory already guarantees the schedules lose money (Section 5.6), the module's stated and only supported use is diagnosing simulator specifications, and it carries that disclaimer in code; the crude, legible schedules it implements are the ones surveillance already catches in real markets, so the probe teaches defenders more than attackers. The one profitable schedule the concave positive control finds (Section 5.8), sliced accumulation and block liquidation, is the textbook Huberman-Stanzl construction and has been in the published literature for two decades.

8 Reproducibility statement

The environment is a Rust workspace of three crates under the MIT and Apache-2.0 licenses, published to crates.io, npm and PyPI, with a pinned toolchain and a lockfile, depending on the published SharpeBench engine crates rather than a vendored copy. The core forbids unsafe code. The determinism claims are enforced rather than asserted: committed FNV-1a golden hashes pin the generated scenarios and the matching engine's fill tape, the WebAssembly crate's tests assert byte-identity with the native engine for both stepping and scenario generation, and the npm package's smoke test performs the replay-and-tamper check of Section 3.3 on every run. The contract artifacts, schemas, conformance fixtures and the governance document, are committed beside the code they govern, and a conformance test exercises the fixtures. Continuous integration covers the Rust surface with formatting, lints as errors, tests and a WASM target build, plus dependency auditing, the npm package and the Python wheel.

Every number in Section 5 is produced by a committed script under `paper/src/` from fixed seeds, writing JSON evidence to `paper/evidence/` and figures to `paper/figures/`; the commands are listed in Section A. The scripts import the published Python package and call only its public API, so reproducing the evidence requires the package version pinned in Section A and nothing else.

No large language model is a component of the environment, the contract or any reported result. The environment has no judge. Language models were used to draft and edit prose; every bibliography entry was checked against its source.

References

Franklin Allen and Douglas Gale. Stock-price manipulation. *The Review of Financial Studies*, 5(3): 503–529, 1992.

Robert Almgren and Neil Chriss. Optimal execution of portfolio transactions. *Journal of Risk*, 3(2): 5–39, 2001.

Robert Almgren, Chee Thum, Emmanuel Hauptmann, and Hong Li. Direct estimation of equity market impact. *Risk*, 18(7):58–62, 2005.

Selim Amrouni, Aymeric Moulin, Jared Vann, Svitlana Vyetrenko, Tucker Balch, and Manuela Veloso. ABIDES-Gym: Gym environments for multi-agent discrete event simulation and application to financial markets. In *Proceedings of the Second ACM International Conference on AI in Finance (ICAIF)*, 2021.

Marco Avellaneda and Sasha Stoikov. High-frequency trading in a limit order book. *Quantitative Finance*, 8(3):217–224, 2008.

David H. Bailey and Marcos López de Prado. The deflated sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality. *The Journal of Portfolio Management*, 40(5):94–107, 2014.

Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI Gym, 2016.

Eric Budish, Peter Cramton, and John Shim. The high-frequency trading arms race: Frequent batch auctions as a market design response. *The Quarterly Journal of Economics*, 130(4):1547–1621, 2015.

David Byrd, Maria Hybinette, and Tucker Hybinette Balch. ABIDES: Towards high-fidelity multi-agent market simulation. In *Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation (SIGSIM-PADS)*, 2020.

Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.

Rama Cont. Empirical properties of asset returns: Stylized facts and statistical issues. *Quantitative Finance*, 1(2):223–236, 2001.

Farama Foundation. Minari: A dataset API for offline reinforcement learning. `https://minari.farama.org`, 2024.

J. Doyne Farmer. Market force, ecology and evolution. *Industrial and Corporate Change*, 11(5): 895–953, 2002.

Jim Gatheral. No-dynamic-arbitrage and market impact. *Quantitative Finance*, 10(7):749–759, 2010.

Lawrence R. Glosten and Paul R. Milgrom. Bid, ask and transaction prices in a specialist market with heterogeneously informed traders. *Journal of Financial Economics*, 14(1):71–100, 1985.

Olivier Guéant, Charles-Albert Lehalle, and Joaquin Fernandez-Tapia. Dealing with the inventory risk: A solution to the market making problem. *Mathematics and Financial Economics*, 7(4): 477–507, 2013.

Gur Huberman and Werner Stanzl. Price manipulation and quasi-arbitrage. *Econometrica*, 72(4): 1247–1275, 2004.

Joseph Jerome, Leandro Sánchez-Betancourt, Rahul Savani, and Martin Herdegen. Mbt-gym: Reinforcement learning for model-based limit order book trading. In *Proceedings of the Fourth ACM International Conference on AI in Finance (ICAIF)*, 2023.

Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized level replay. In *Proceedings of the 38th International Conference on Machine Learning*, 2021.

Albert S. Kyle. Continuous auctions and insider trading. *Econometrica*, 53(6):1315–1335, 1985.

Xiao-Yang Liu, Hongyang Yang, Qian Chen, Runjia Zhang, Liuqing Yang, Bowen Xiao, and Christina Dan Wang. FinRL: A deep reinforcement learning library for automated stock trading in quantitative finance, 2020. Deep RL Workshop, NeurIPS 2020.

Xiao-Yang Liu, Ziyi Xia, Jingyang Rui, Jiechao Gao, Hongyang Yang, Ming Zhu, Christina Dan Wang, Zhaoran Wang, and Jian Guo. FinRL-Meta: Market environments and benchmarks for data-driven financial reinforcement learning. In *Advances in Neural Information Processing Systems 35, Datasets and Benchmarks Track*, 2022.

Thomas Lux and Michele Marchesi. Scaling and criticality in a stochastic multi-agent model of a financial market. *Nature*, 397:498–500, 1999.

Marlos C. Machado, Marc G. Bellemare, Erik Talvitie, Joel Veness, Matthew Hausknecht, and Michael Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61:523–562, 2018.

Shuo Sun, Molei Qin, Wentao Zhang, Haochong Xia, Chuqiao Zong, Jie Ying, Yonggang Xie, Lingxuan Zhao, Xinrun Wang, and Bo An. TradeMaster: A holistic quantitative trading platform empowered by reinforcement learning. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023.

Jordan Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis S. Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, Niall Williams, Yashas Lokesh, and Praveen Ravi. Pettingzoo: Gym for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

Tiberiu Toca. SharpeBench: A Luck-Robust Benchmark for Trading Agents. Preprint, `https://github.com/general-liquidity/sharpebench`, 2026. Version 0.6.0.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments, 2024.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.

Gilles Zumbach. Time reversal invariance in finance. *Quantitative Finance*, 9(5):505–515, 2009.

A Commands that produce every number

All commands run from the repository root with the `sharpearena` Python package installed. The original eight evidence files record `sharpearena` 0.8.0 over SharpeBench 0.5.0. The volatility-clustering remediation shipped at 0.10.0; the concave-impact and predictability extensions are versioned with the 0.11.0 source tree; the positive control, the existence witness and the sealed seeds were produced on the post-0.11.1 tree. Each script fixes and serializes its seeds.

The design-property checks (Sections 3.1 to 3.3 and 4).

```
cargo test -p sharpearena              # golden hashes, leak-free
                                       # property tests, conformance kit
cargo test -p sharpearena-wasm         # native == wasm parity + golden
npm test --prefix npm/sharpearena      # replay + tamper detection
python -m pytest crates/sharpearena-py # Python surface, check_env,
                                       # seed-band disjointness
```

F1: baseline leaderboard (Section 5.1).

```
python paper/src/make-f1-baselines.py
  -> paper/evidence/f1-baselines.json, paper/figures/f1-baselines.pdf
```

The script passes the bootstrap parameters explicitly ($B = 2000$ percentile resamples, $\alpha = 0.05$, resample seed `0x5BA7_2026`) and serializes them in the evidence config; they equal the library defaults defined in `crates/sharpearena-py/python/sharpearena/confidence.py` (`DEFAULT_N_BOOT`, `DEFAULT_RESAMPLE_SEED`), which reuse the scoring kernel's canonical bootstrap seed.

F2: regret vs the closed-form optimum (Section 5.3).

```
python paper/src/make-f2-regret.py
  -> paper/evidence/f2-regret.json, paper/figures/f2-regret.pdf
```

F3: generalization gap and cross-regime transfer (Section 5.4).

```
python paper/src/make-f3-generalization.py
  -> paper/evidence/f3-generalization.json,
     paper/figures/f3-transfer-matrix.pdf
```

F4: stylized-facts realism certification (Section 5.5).

```
python paper/src/make-f4-realism.py
  -> paper/evidence/f4-realism.json, paper/figures/f4-realism.pdf
```

F5: manipulation boundary and size response (Section 5.6).

```
python paper/src/make-f5-manipulation.py
  -> paper/evidence/f5-manipulation.json,
     paper/figures/f5-boundaries.pdf, paper/figures/f5-size-response.pdf
```

The same command also runs the concave-impact extension at $\kappa = 0.5$ and 0.7, serializes it under `concave`, and writes `paper/figures/f5-concave.pdf` (Section 5.7). It then runs the asymmetric positive control (Section 5.8): 135 grid points over five duration ratios, three block fractions and $\kappa \in \{1.0, 0.7, 0.5\}$ on the theory, temporary-impact and canonical arms, eight seeds each, serialized under `positive_control` with its `config` (including the `not_searched` list) and written to `paper/figures/f5-positive-control.pdf`. The `linear` and `concave` keys of the evidence file are byte-identical to their previous values.

Rank-eligibility existence witness (Section 5.2).

```
python paper/src/make-witness.py
  -> paper/evidence/witness.json, paper/figures/witness.pdf
```

The command replays each seed one bar ahead to recover the true next-bar return (asserting that the canonical tape is policy-invariant), mixes it with seeded noise at 13 coarse strengths, scores `sign_follow` and `deadband_hold` through `score_run` on every seed and on the pooled series, and bisects each tier's boundary to a resolution of 0.005 in s on the held-out band and the F1 table band. The JSON records every strength's per-seed PSR, pooled DSR, bootstrap p-value, eligibility flag and the bracketed boundaries behind Table 2.

Sealed evaluation seeds (Section 5.13).

```
python paper/src/make-sealed-seeds.py
  -> paper/evidence/sealed-seeds.json
```

The command derives 16 named slots publicly (the eight committed offsets plus eight drawn from the same 2^{16} band) and sealed (the same slots under a 32-byte salt from the operating system, committed by SHA-256 before the run), re-runs the predictability probe's band-scan adversary unchanged against both, then reveals the salt and replays the sealed scenarios against the recomputed seeds. The JSON records the commitment, the revealed salt, the table-build time and every per-trial recovery and replay record.

F6: adverse-selection markouts (Section 5.9).

```
python paper/src/make-f6-adverse-selection.py
  -> paper/evidence/f6-adverse-selection.json,
     paper/figures/f6-markouts.pdf
```

F7: failure-mode distributions (Section 5.10).

```
python paper/src/make-f7-failures.py
  -> paper/evidence/f7-failures.json, paper/figures/f7-failures.pdf
```

F8: ecology under shocks, 8 seeds per schedule (Section 5.11).

```
python paper/src/make-f8-ecology.py
  -> paper/evidence/f8-ecology.json,
     paper/figures/f8-ecology-control.pdf,
     paper/figures/f8-ecology-shocked.pdf
```

The evidence carries the full seed-0 reports (the figures' source) plus, under `multi_seed`, per-seed winners, outcome counts and the cross-seed winner distributions behind Table 8.

Generator predictability (Section 5.12).

```
python paper/src/make-predictability.py
  -> paper/evidence/predictability.json,
     paper/figures/predictability.pdf
```

The command evaluates baseline, causal AR(5) and true-seed oracle predictors on 16 seeds per tier, then times exhaustive recovery in a 2^{16} candidate band. The JSON contains every per-seed score and the measured search timing.

Dispersion layers. The F2, F3, F5 and F6 scripts serialize the per-episode or per-seed vectors behind every reported interval into their evidence JSONs (`per_episode_regret`, `per_seed_returns` with `band_dsr_ci` and `transfer_gap_ci`, `dispersion`, and `per_episode_markout_per_unit` with `gap_stats` respectively), each with its CI convention recorded in the file, so every interval in Section 5 is recomputable from the committed evidence.

Regenerating every figure from the committed evidence.

```
python paper/src/make-figures.py
```

Each `make-f*` script produces its own figure at run time; `make-figures.py` re-renders all figures from the JSON already in `paper/evidence/` without re-running any experiment, so a reader can check that no figure contains a number the evidence does not.